

Article

Development of a Knowledge-Distillation-Based Breast Cancer Classifier for LMICs: Comparison with Pruning and Quantization

Falmata Modu ^{1,*}, Rajesh Prasad ^{1,2} and Farouq Aliyu ³¹ Department of Computer Science, African University of Science and Technology, Abuja 900107, Nigeria² Department of Computer Science and Engineering, Ajay Kumar Garg Engineering College, Ghaziabad 201015, India; prasadrajes@akgec.ac.in³ Center of Excellence in Development of Non-Profit Organizations, King Fahd University of Petroleum and Minerals, Dhahran 31261, Saudi Arabia; farouq.muhammad@kfupm.edu.sa

* Correspondence: falmata@aust.edu.ng

Abstract

Breast cancer (BC) mortality rates remain high in Low- and Middle-Income Countries (LMICs) due to limited awareness, poverty, and inadequate medical facilities that hinder early detection. Although deep learning models have achieved high accuracy in BC detection (BCD), they require substantial computational resources, making them unsuitable for deployment in remote or rural areas. This study proposes a lightweight convolutional neural network (CNN) using Knowledge Distillation (KD) for BCD, where a large Teacher Model (TM) transfers learned representations to a smaller Student Model (SM), which is better suited for deployment on low-power devices. We compare it with two prominent model compression techniques: pruning and quantization. Experimental results indicate that the TensorFlow Lite (TFLite)-optimized Student Model (SM_TFLite) achieved 97.67% accuracy, representing a 2.33% relative loss to its teacher, a result comparable to other compression techniques. Its mean accuracy is 73.97% with a 95% Confidence Interval of [65.04%, 82.90%] in a cross-dataset experiment. However, SM_TFLite was the most compact (5.21 kB) and fastest (3.3 ms latency), outperforming both pruned (2924.31 kB, 13.68 ms) and quantized models (746–751 kB, 4–5 ms). Evaluation on a Raspberry Pi 4 Model B demonstrated that all models exhibited similar CPU and memory usage, with SM_TFLite causing only a minor increase in device temperature. These results demonstrate that KD combined with TFLite conversion offers the best trade-off between accuracy, compactness, and speed.



Academic Editor: Hyunjin Park

Received: 4 November 2025

Revised: 6 December 2025

Accepted: 6 December 2025

Published: 9 December 2025

Citation: Modu, F.; Prasad, R.; Aliyu, F. Development of a Knowledge-Distillation-Based Breast Cancer Classifier for LMICs: Comparison with Pruning and Quantization.

Electronics **2025**, *14*, 4842.<https://doi.org/10.3390/electronics14244842>**Copyright:** © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license

<https://creativecommons.org/licenses/by/4.0/>.

Keywords: breast cancer detection; lightweight CNN; Knowledge Distillation; pruning; quantization; IoT; healthcare; telemedicine; breast cancer; deep learning; convolutional neural network; Keras

1. Introduction

The World Health Organization (WHO) defines Breast Cancer (BC) as a life-threatening malignancy characterized by the uncontrolled growth of abnormal cells forming a tumor in the breast, which, if left untreated, can metastasize (spread) to vital organs through the bloodstream and lymphatic system, ultimately becoming fatal [1]. Most cases of BC begin in the ducts that carry milk to the nipple or in the lobules that produce milk. These changes often result from genetic mutations that activate growth-promoting genes or disable tumor-suppressing ones, allowing cells to divide unchecked. While some mutations, such as

in the BRCA1, BRCA2, or PALB2 genes, are inherited, most occur spontaneously during a person's lifetime [2]. BC is not a single disease but a group of disorders originating from different cells within the breast tissue. The cancer cells can spread to lymph nodes and distant organs such as the lungs, liver, brain, or bones, as the tumor progresses from localized (in situ) to invasive and eventually metastatic stages. Therefore, BC can become potentially fatal if not detected and treated early. Table 1 summarizes the significant forms of breast cancer and their diagnosis technique [1–3].

Table 1. Summary of the Different Types of Breast Cancer.

BC Type	Common Location	Prevalence	Diagnosis
Ductal Carcinoma in Situ	Milk ducts	~15–25%	Mammography, biopsy
Invasive Ductal Carcinoma	Milk ducts	~70–80%	Physical exam, mammography, biopsy
Invasive Lobular Carcinoma	Lobules	~10%	MRI, ultrasound, biopsy
Triple-Negative Breast Cancer	Usually ducts	~15%	Immunohistochemistry, biopsy
Inflammatory Breast Cancer	Lymph vessels and ducts	~1–5%	Clinical symptoms, imaging, biopsy
Paget Disease of the Breast	Nipple and ducts	~1–3%	Nipple skin exam, biopsy
Phyllodes Tumor	Connective tissue (stroma)	Rare	Physical exam, imaging, biopsy
Angiosarcoma of the Breast	Blood or lymph vessels	less than 1%	Biopsy, histopathology

A recent global analysis of 2022 BC data from GLOBOCAN, Cancer Incidence in Five Continents Plus (CI5plus), and the WHO Mortality Database reported 2.3 million new cases and 670,000 deaths, accounting for 25% of all new cancer diagnoses and 15% of cancer-related deaths among women, respectively [4]. The study found that incidence rates were highest in Australia and New Zealand (approximately 100 cases per 100,000 women), followed by Northern America and Northern Europe, and lowest in South-Central Asia, Middle Africa, and Eastern Africa (below 30 cases per 100,000). These regional differences show variations in lifestyle choices, productive habits, and access to healthcare. On the one hand, countries with a high Human Development Index (HDI) report higher counts due to factors such as obesity, alcohol consumption, delayed childbirth, lower fertility rates, and increased case identification through various screening programs. On the other hand, Low- and Middle-Income Countries (LMICs) record fewer cases of breast cancer. However, they experience higher death rates due to late diagnosis and limited access to treatment options [4]. The reports also reflect that, by the next 25 years, BC cases are likely to rise by nearly 40% and deaths by almost 70%, with the steepest increases in LMICs. If early detection and treatment do not improve, these growing differences may make it harder to lower breast cancer deaths worldwide.

Researchers are increasingly using Artificial Intelligence (AI) to improve early Breast Cancer Detection (BCD). One U.S. study [5] compared an AI-based BCD system to ten board-certified radiologists. The radiologists reviewed breast ultrasound images with complete clinical information, while the AI system had only the images. Even with this difference, the AI performed better, showing higher diagnostic accuracy (AUROC = 0.962, AUPRC = 0.752), greater sensitivity (94.5%) and specificity (85.6%), a 4.5% reduction in unnecessary biopsies, and a 5.4% increase in positive predictive value (PPV). These findings show that AI can detect BC more accurately and reduce false positives. Similarly, another extensive study in Germany [6] involving over 460,000 women at 12 screening sites also showed that AI-supported double reading led to a 17.6% higher BCD rate than standard screening, with a similar or slightly lower false positive rate. The AI system also increased the PPV of both recalls and biopsies, detecting more true cancers per procedure, and reduced radiologists' workload by up to 56.7% through efficient triaging of normal mammograms.

However, the adoption of technology in healthcare remains limited across LMICs. Figure 1 presents a heatmap illustrating the state of digital health maturity according to World Bank income groups in 2023 [7]. The findings demonstrate a strong correlation between digital health maturity and national income levels. High-income countries tend to fall within the most advanced stages (phases 4–5), where digital health systems are already well established. In comparison, many upper- and lower-middle-income nations are still developing, usually positioned around the middle phases (2–4). Most low-income countries are at the early stages (phases 1–2), which shows that digital infrastructure is still limited and that the use of digital technologies, including AI for breast cancer detection, is progressing more slowly. Many LMICs still struggle with poor internet connectivity, limited access to modern imaging tools, and a shortage of trained healthcare workers [8]. Financial barriers make the situation even more difficult, as setting up, maintaining, and upgrading these systems can be costly [9]. Addressing these challenges is key to ensuring that AI-based BCD becomes more accessible and equitable in healthcare settings with limited resources.

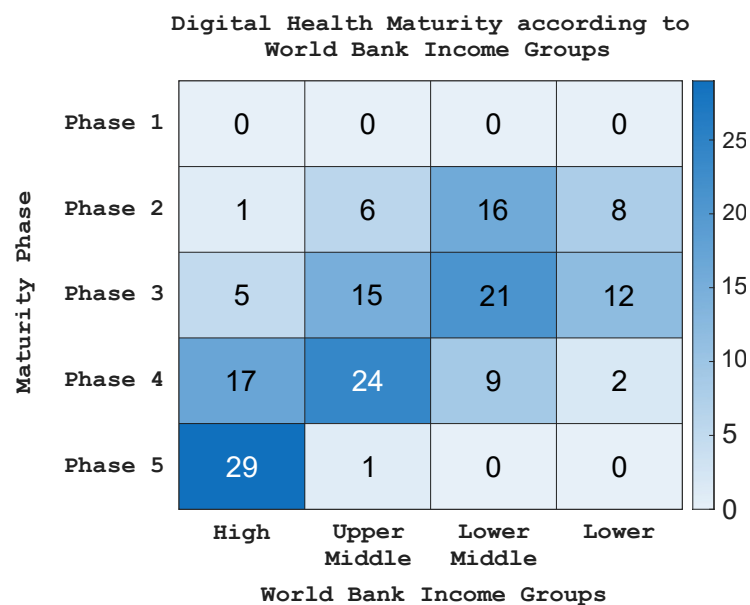


Figure 1. Heatmap of WHO’s Digital Health Maturity Phase according to the World Bank Income Groups in 2023.

Lightweight Machine Learning (ML) for BCD can help increase early detection rates in LMICs. An ML model is lightweight if it possesses low computational requirements, a small model size, a low memory footprint, and high energy efficiency [10]. These features make it suitable for real-time-based classification systems for resource-constrained environments. Deploying such models on low-cost Single-board Computers (SBCs) such as the Raspberry Pi (RPi) can help address the primary challenge of implementing AI in LMICs. These models require minimal computational power and can operate offline, addressing infrastructure limitations such as unreliable internet and limited hardware. Their low cost and energy efficiency reduce financial burdens, while their portability enables deployment in remote or underserved areas. Moreover, lightweight ML systems are easy to maintain. They can assist healthcare workers with limited AI expertise, making them a practical solution for improving access to BCD in resource-constrained settings.

This paper presents a lightweight Convolutional Neural Network (CNN)-based BCD model using the Knowledge Distillation (KD) technique. KD is one of the approaches used to develop lightweight Deep Learning (DL) models by transferring knowledge from a pre-trained deep CNN (the Teacher Model (TM)), to a smaller/shallow model (the Student Model (SM)). This technique allows the SM to learn key features from the TM

while maintaining high accuracy and energy efficiency. In our previous work [11], we developed a lightweight CNN for BCD using KD, achieving an 87% reduction in trainable parameters while maintaining high performance (99.3% accuracy, 100% precision, and 99% recall). Building on that, this study further improves the SM with the TensorFlow Lite (TFLite) framework, creating the SM_TFLite model that offers faster inference, smaller file size, and better deployment efficiency than pruning and quantization techniques. These improvements make lightweight AI more practical for BCD in IoT-based healthcare systems. The main contributions of this paper are summarized as follows:

1. Design and implementation of a KD-based lightweight CNN model that achieves high detection accuracy while maintaining a significantly reduced model size, making it suitable for deployment on resource-constrained devices.
2. Extensive evaluation across three diverse BC datasets, demonstrating consistently high performance, robustness, and generalization capability.
3. Cross-dataset evaluation of the proposed model by training it on one dataset and applying it to another with different feature distributions, showing the proposed model outperforms the SM without KD despite the expected accuracy drop under domain shift.
4. Comparative analysis of the proposed KD-based SM with other lightweight conversion techniques, including TFLite conversion, pruning, post-training quantization (PTQ), and quantization-aware training (QAT), extending our previous work in [11].
5. Preliminary deployability assessment on a RPi 4 Model B, confirming low memory footprint and low-latency inference, underscoring its potential for real-time telemedicine applications in resource-limited environments.

The remaining parts of this paper are as follows: Section 2 reviews related work on lightweight ML-based BCD systems identified from the Scopus database. Section 3 presents a short background on KD, PTQ, and QAT techniques, as well as the datasets, Primary Breast Cancer vs. Normal Breast Tissue (PBCNT), Breast Cancer Dataset (BCDD), and Breast Cancer Wisconsin (Diagnostic) Dataset (WDBC), which form the foundation of this study. Section 4 describes the proposed model and its design. Section 6 presents the experimental setup and performance evaluation of the model. Section 7 discusses the limitations of the proposed model. Finally, Section 8 concludes the paper and outlines potential directions for future research.

2. Related Works

In this section, we review the existing lightweight BCD-based ML methods. There is no generally accepted definition of lightweight ML. Thus, different authors have put forward different criteria. Li et al. [12] define Lightweight ML as a technique that can run on a standard computer with a single processor core. In contrast, Kim et al. [13] define it as a model that can run on resource-constrained IoT devices without reducing system performance. We will use the latter definition in this paper, where a resource-constrained device is one with limited memory and computational power. Table 2 shows the summary of the performance of the models investigated in the literature review.

Many studies have focused on creating lightweight models to detect BC with the help of transfer learning, architecture adjustments, or multi-stage approaches: Ahmed et al. [14] used DenseNet201 in a mammogram analysis study with fine-tuning, which indicates that pre-trained models can be successfully tailored to medical imaging tasks at only a small amount of computational resources. These approaches highlight the trend toward combining efficient architectures, such as MobileNet, with transfer learning to create practical, low-latency diagnostic tools. Pramanik et al. [15] employed SqueezeNet 1.1 combined with genetic and grey wolf optimizers for feature selection. Whereas Laxmisagar et al. [16,17]

used MobileNet for feature extraction followed by a CNN classifier, and further enhanced the pipeline with stain-removal preprocessing. Similarly, Ahmad et al. [14] demonstrated that data augmentation significantly improves the performance of MobileNetV1, ResNet50, and DenseNet201. Abdelli et al. [18] further showed that combining the BreakHis and IDC datasets boosts model performance. Other lightweight variants integrate auxiliary networks or simplified CNNs; for instance, Liu et al. [19] used a U-net for Region of Interest (ROI) localization together with a CNN for prediction, Oliveira et al. [20] combined ResNet50 and VGG16 with contour refinement, and Elbashir et al. [21] designed a compact CNN with only two convolutional and two dense layers.

Table 2. Performance of Lightweight Methods in the Literature.

Ref.	Year	Dataset	Technique	Acc. (%)	Prec. (%)	Recall (%)	F1-Score
[12]	2018	ICPR2012	Random forest + CNN		78	79	0.784
[21]	2019	TCGA	CNN	98.76	100	91.42	0.955
[20]	2019	INbreast	ResNet50 + VGG16			85	
[18]	2020	BreakHis, IDC	MobileNet ResNet50	94.42 97.03	94.99 97.41	94.42 97.03	0.9449 0.9705
[19]	2020	Camelyon16	U-net + CNN				
[16]	2021	BACH	MobileNet2.10ex + CNN	88.92			
[22]	2022	BreakHis	LightXception (Averaging)	96.89	99.1	96.35	0.977
			LightXception (Threshold)	97.69	98.15	98.54	0.9833
			LightXception (Whole)	96.59	97.85	97.23	0.9751
			Xception (Averaging)	99	99.86	98.69	0.9926
			Xception (Threshold)	99.05	99.86	98.76	0.993
			Xception (Whole)	99.7	99.78	99.78	0.9978
[17]	2022	BACH	MobileNet1.40 + CNN	87.5			
			MobileNet1.40ex + CNN	72.91			
			MobileNet2.00 + CNN	75			
[15]	2023	DMR-IR	SqueezeNet 1.1 + GA + GWO + k-nn	100	100	100	1
[23]	2023	IDC	MobileNetV2 + CNN	92.3	84.09	88	0.86
		BreakHis	MobileNetV2 + CNN	98.56	96.42	98.18	0.9729
		BACH	MobileNetV2 + CNN	92	92.16	91.94	0.9215
[24]	2023	BreakHis	MobileViT	95.3	98.8	99.8	0.969
[25]	2023	BreakHis	Separable CNN	97.23	98.11	97.71	0.9798
[26]	2023	ICIAR	Deep CNN + Separable Conv	96.25	99.8	99.4	0.995
		BreakHis	Deep CNN + Separable Conv	99.4			
		Brac	Deep CNN + Separable Conv	72.2	75.8	74.4	0.755
[27]	2024	BreakHis	PCSAM + ResCBAM	98.74	99	99	0.99
		INCIAR2018	PCSAM + ResCBAM	97.5	97.5	97.62	0.975
[28]	2024	Not Available	MD-VACNet	98.47	98.31	98.42	
[29]	2024	Proprietary	CNN	85	81.98	97.88	0.8923
[30]	2020	PUSH Ultrasound	CNN + KD		96.33	89.25	
[12]	2018	ICPR 2012	Random forest + CNN		78	79	
[31]	2023		Self-supervised KD	85			
[32]	2023	BreakHis	Resnet-50 + Quantization aware	94.26			
			VGG19 + Quantization aware	94.81			
[33]	2024		KD	90.63	99.68	98.74	99.21

In addition to architectural optimization, researchers have also achieved complexity reduction through KD, separable convolutions, or hybrid learning strategies. Zhang et al. [30] proposed a lightweight CNN to classify ultrasound-based lesions. They applied KD to train their model as the SM using YOLOv3 as the TM. They also employ an asynchronous calculation strategy that processes only a quarter of the network per image frame, reducing the model's computational power requirements. Wang et al. [34] employed KD with EfficientNet to train a lightweight student model for histopathology image classification, significantly lowering memory requirements while maintaining strong accuracy. Li et al. [12] proposed a multi-stage framework using random forests and CNNs, and Ashraf et al. [35] combined a shallow ResNet50 with an Inception module via self-supervised contrastive learning. Scientists have also investigated ensemble and federated learning approaches, such as MobileNetV2 with a shallow CNN in Garg et al. [23] and federated learning with MobileViT in Yan et al. [24]. Additional lightweight designs include CNNs with separable and bottleneck convolutions [25,26], SqueezeNet-based transfer learning [36], patch-level feature fusion networks [27], and mobile-integrated DL systems for cytology images [29].

The literature shows that pruning and quantization reduce redundancy in CNN models, making them lighter and deployable on edge devices with limited capacity [37]. Kim et al. [22] introduced LightXception. The technique prunes the original Xception model, removing 65% of its layers and convolutional filters while maintaining an accuracy of up to 97.69%. Kumar et al. [38] applied Post-Training Quantization (PTQ) to MobileNetV2 for histopathological image classification to form MobiHisNet. They achieved an 84.09% reduction in model size (from 12.82 MB to 2.04 MB) while increasing accuracy by up to 3 percentage points. However, when they compressed the model further, reducing it by 91.73% of its original size, the accuracy fell by up to 22 percentage points. Another study [32] successfully deployed VGG19- and ResNet-50-based models optimized using joint sparsity Quantization-Aware Training (QAT) on embedded systems.

Yan et al. [27] integrated attention mechanisms into lightweight models to improve feature selection without substantially increasing computational costs. Similarly, Nissar et al. [39] improved the MobileNetV3 framework by incorporating channel attention to enhance feature representation. Their technique achieved a 99% accuracy, precision, and recall. However, adding the Convolutional Block Attention Module (CBAM) would increase the model size and complexity. Another study proposed a Multi-Disease Visual Attention Condenser Network (MD-VACNet) with Self-attention and spatial pyramid pooling [28] for edge computing devices. The self-attention is a mechanism that allows a model to weigh the importance of different spatial or temporal features relative to one another. Hence, it allows MD-VACNet to localize cancer features and improve the model performance. The authors reported a 98.99% accuracy with 25% fewer parameters than MobileNet-V1.

Despite the aforementioned advances in lightweight BCDs, several challenges remain: Some models [18,20,22,40] claim to be lightweight, yet their parameter counts remain too large for efficient deployment on low-resource devices such as the RPi and mobile phones. Others [16,17,19,29,41] reduced model size at the cost of diagnostic accuracy. Also, many studies rely on a single dataset or lack evidence of real-world deployability. Furthermore, while pruning and quantization have been widely studied in other domains, to the best of our knowledge, there are no systematic comparative analyses with KD in the context of BCD. This paper addresses these gaps by developing a KD-based lightweight CNN for BCD and comparing it against pruning- and quantization-based models within a unified evaluation framework. It tested the models across multiple benchmark datasets. It also evaluated their inference latency, memory footprint, and power consumption on a RPi 4 Model B, assessing the model's deployability on resource-constrained devices.

3. Background

This section presents the foundations of this paper. It introduces the concept of KD. It discusses the basics of pruning and quantization. It also presents three key text-based datasets used in our research: the PBCNT, BCDD, and WDBC. These datasets help us investigate the effectiveness of KD techniques in improving BCD.

3.1. Knowledge Distillation

KD is an ML compression technique that transfers knowledge from a large model to a smaller one [42]. The large model is known as the Teacher model (TM), while the small one is called the Student Model (SM). In other words, KD distills the TM's knowledge and stores it in the SM. Conventional ML technique tries to fit a neural network to the training data, while KD aims to train an SM to replicate the predictions of a TM [43]. Thus, it finds application where large models cannot be deployed due to resource constraints, necessitating compression by transferring their knowledge to a smaller model [43,44].

KD offers several benefits. It increases system speed and efficiency by reducing model size and complexity. Thereby, allowing deployment on resource-constrained devices, such as mobile phones, SBCs, and drones [45]. It can also transfer knowledge without using the complete dataset, improving data security and privacy [42]. This method has been successfully applied across various fields, including natural language processing, speech recognition, and image recognition [43]. Additionally, KD finds application in scenarios where rapid inference is crucial, such as real-time applications in autonomous vehicles or real-time video analysis, where the speed and efficiency of the model are critical [45].

Pytorch and Keras have KD libraries [45,46]. This paper uses the Keras implementation [46]. The technique minimizes a loss function that incorporates both the teacher's softened logits and the ground-truth labels during training. Figure 2 is the flowchart for teaching the SM using KD. Intel® Labs inspired it [44]. The process starts with inputting the dataset features (X) into a pre-trained TM and the SM. The TM and the SM produce the z_{TM} and the z_{SM} logits, respectively. KD also collects the softmax Temperature (T) and weighted average parameter (α) from the developer. It uses the softmax function (see Equation (1) proposed by Hinton et al. [47] to create TM soft labels, SM soft predictions, and SM hard predictions as q_{TM} , q_{SM} , and q'_{SM} , respectively.

Hinton et al. [47] introduced the parameter T into the softmax function to obtain information beyond the dataset's ground-truth labels (y). As T grows, the softmax function's probability distribution becomes softer, providing more information about the teacher's prediction. The authors tested T values ranging from 1 to 20. They found that lower values work better when the SMs are lighter than the TMs, because higher temperatures may provide too much information for the student to capture effectively. However, Intel® Labs argues that there are no methods for predicting the capacity of information the SM acquires.

$$\text{softmax}(z_i, T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (1)$$

Where

z_i = logits for class i ,

T = temperature parameter.

Our proposed model uses Keras's Kullback–Leibler (KL) divergence loss function (H_{KLD}) to minimize the squared difference between the soft logits of the TM and SM (i.e., q_{TM} and q_{SM} , respectively). It also uses the Binary Cross Entropy loss function (H_{BCE}) to calculate the Loss between the SM's hard prediction (q'_{SM}) and the ground-truth (y). In the last step, the system uses a weighted average on H_{BCE} and H_{KLD} losses with α as the

weighing parameter to calculate the SM's Loss ($Loss_{SM}$). Hinton et al. [47] typically used a small α in their weighted average. Smaller α values enable the student model to place greater emphasis on the teacher's softened predictions, thereby improving generalization.

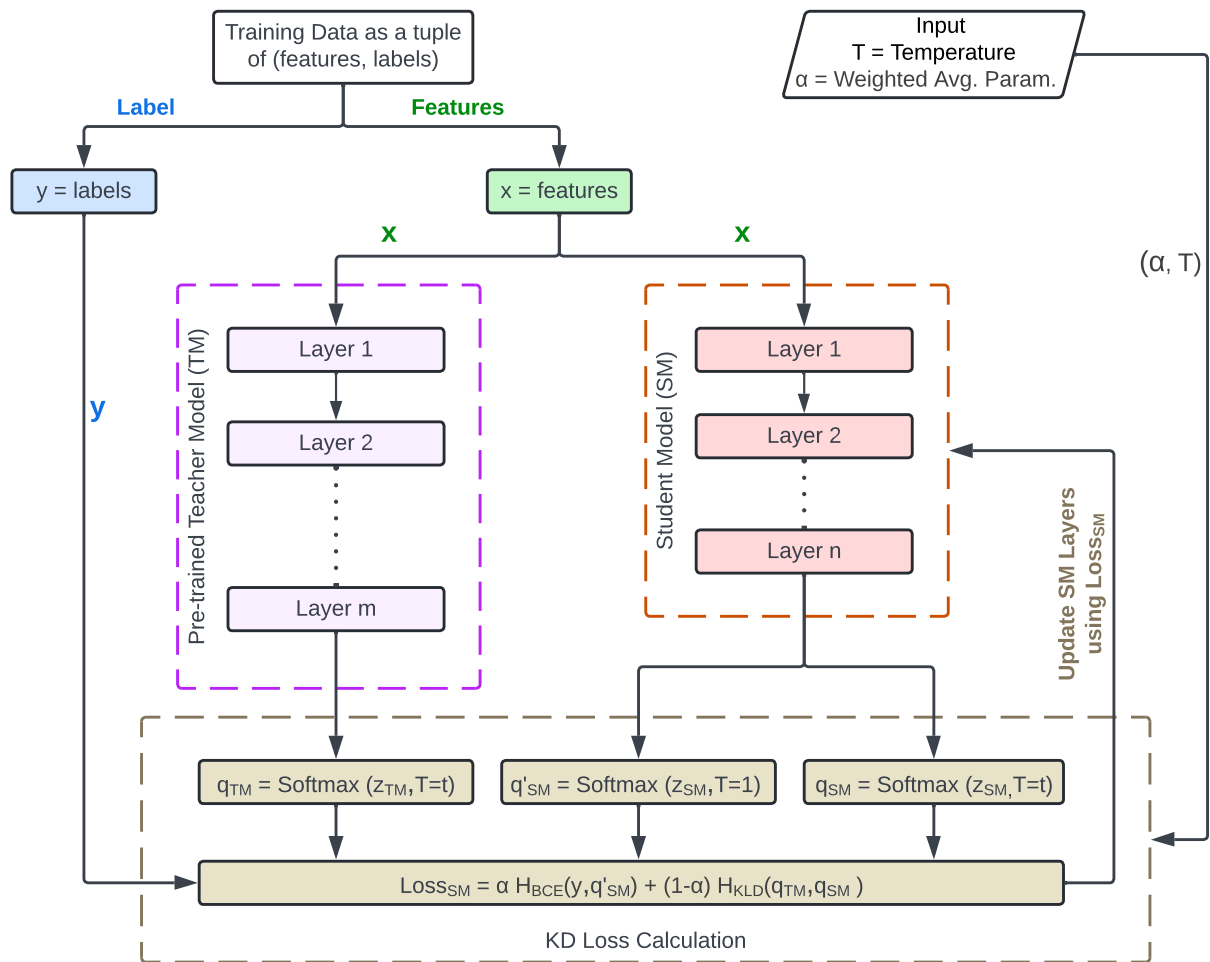


Figure 2. SM Learning Process using KD.

In this paper, we used $\alpha = 0.1$ and $T = 3$ for the KD hyperparameters. Section 6.1 justifies the selection of the aforementioned values through an ablation study, which evaluates the influence of different α and T combinations on model performance.

3.2. Pruning

Pruning is a process that removes unnecessary parts of an ML model, simplifying and reducing its size. This paper adopts the definition of Vadera et al. [48], which defines pruning as the process of reducing the size and complexity of a trained neural network by removing redundant or less important parameters, such as weights, neurons, filters, or channels, while maintaining model accuracy. It is used in Deep Neural Networks (DNNs) to reduce size and latency, in Decision Trees to prevent overfitting, in Random Forest to simplify ensembles, and in Rule-Based models to improve interpretability.

There are several forms of pruning in neural networks, such as structured pruning, which removes entire filters or channels, and unstructured (magnitude-based) pruning, which removes individual weights. In this paper, we adopt magnitude-based pruning because it is practical, widely used, and easily transferable across models [48]. We applied the low-magnitude pruning technique from the TensorFlow model optimization framework [49], which automatically increases the sparsity of the model's layers during training by setting low-importance weights to zero. Alternatively, users can specify the desired

sparsity level. For example, assigning a target sparsity of 50% forces half of a layer's weights to become zero, reducing model complexity while preserving the network structure. This pruning mechanism is versatile, as it can be applied to individual layers, groups of layers, or entire models. However, it raises an error when it encounters unsupported or unrecognized layer types.

3.3. Quantization

Quantization is the conversion of high-precision values in weights or activations into lower-precision ones. It converts real numbers (float or double data types) to integer values (INT8 or INT4 data types) [50]. DDNs are tolerant to precision reduction due to their over-parameterized structure, which allows them to maintain accuracy even when many parameters are quantized [50]. The quantization function in Equation (2) expresses the transformation process:

$$Q(r) = \text{Int}\left(\frac{r}{S}\right) - Z, \quad (2)$$

where r is a real-valued parameter or activation, S is a scaling factor, and Z is a zero-point used for mapping real numbers to integers.

Quantization offers several advantages for efficient deployment of DNN on resource-constrained devices [50–52]: It reduces model size by replacing 32-bit floating-point values with 8-bit integers, achieving up to a $4\times$ smaller memory footprint and enabling deployment on embedded or edge hardware. It accelerates inference by up to $4\times$ due to faster integer computations and lower memory bandwidth requirements. Quantization also improves energy efficiency, delivering up to $16\times$ better performance per watt, extending battery life for edge devices and reducing operational costs in cloud environments. Modern AI accelerators such as Google Edge TPU and NVIDIA Tensor Cores are optimized for integer arithmetic, allowing quantized models to utilize available hardware resources while maintaining near-original accuracy fully.

Several quantization approaches exist, each with different implementation complexity, accuracy trade-offs, and deployment constraints. In this work, we adopt PTQ and QAT. PTQ provides fast, low-cost model compression suitable for real-time or embedded systems, whereas QAT is essential when maintaining diagnostic reliability is critical and minor accuracy losses are unacceptable [50]. The following subsections introduce the specific PTQ and QAT techniques we used.

3.3.1. Weight-Based and Full Integer PTQ

PTQ can be weight-only, dynamic-range, or full-integer quantization [53]. Weight-only quantization converts only the weights to integers (INT8), while the remaining parameters, such as activations, inputs, and outputs, remain in real numbers (FLOAT32). The integer computations of the weights reduce latency. However, the technique needs quantization and dequantization steps for integer-based data to work correctly with floating-point data.

The dynamic range quantization technique is similar to its weight-only counterpart. It converts weights to integers while leaving other tensors in their real number format. However, it dynamically quantizes activations to integers at runtime, enabling computations between weights and activations to be performed in integer format. This technique does not fully compress the model, since it stores activations in floating-point format (i.e., real numbers). Furthermore, this technique requires an additional dequantization operation since the input and output tensors also remain in floating-point format.

Full-integer PTQ converts all floating-point tensors, including inputs and outputs, into integers. This technique leads to the highest reductions in model size and improvements in inference speed [53]. The technique is useful for devices that support only integer values, such as 8-bit microcontrollers and the Coral Edge TPU. However, the range of values

for variable tensors must be determined by running computations with a representative dataset for calibration.

In this paper, we adopt weight-only and full-integer PTQ to evaluate the two extremes of this technique. The TFLite Converter was used to implement these approaches [54]. We compare PTQ results with our KD model in Section 6.5.

3.3.2. Quantization-Aware Training (QAT)

QAT quantizes the model during training by simulating low-precision (integer) arithmetic. It introduces fake quantization operations on the weights and activations and uses the results to optimize the quantization parameters [55]. In other words, QAT quantizes the model during training to enable it to optimize its parameters under the given low-precision constraints. This process allows QAT to maintain the model's accuracy and integrity despite reduced precision. However, it has a higher computational overhead than PTQ.

We implemented QAT and compared its performance with the proposed KD model in Section 6.6. It was implemented using custom quantizers and layer-level annotations from the TensorFlow model optimization framework [56]. This technique produces a lighter model than other QAT methods by using special wrappers. The wrappers are used during training to introduce the fake quantization operations we mentioned earlier, allowing the DNN to adapt to 8-bit quantization constraints. After the training, the final model was further optimized by converting it to TFLite format.

3.4. Datasets

This paper used the PBCNT, BCDD, and WDBC datasets to test the performance of the proposed model. Using multiple datasets enabled us to assess the model's generalization and robustness. This section describes the datasets used in this paper to test the model.

3.4.1. Primary Breast Cancer vs. Normal Breast Tissue Dataset (PBCNT)

The PBCNT dataset was generated using microarray-based profiling to compare primary breast cancer tissues with normal breast tissue based on tumor signature miRNA profiles [57,58]. The dataset contains 133 rows and 1928 columns. The first 1926 columns represent normalized microRNA (miRNA) expression features, while the last two columns, named `target` and `target actual`, encode the class labels: `target` contains a binary value (1 = primary breast cancer, 0 = normal breast tissue), and `target actual` contains the textual class names (primary breast cancer, or normal breast tissue). The sample distribution is skewed, with 122 (91.7%) as normal breast tissue samples and 11 (8.3%) as primary BC samples. Some features correspond to negative control probes (e.g., `hsa_negative_control_6-8`) for internal calibration and experimental validation.

This dataset can be used for supervised learning and binary classification, enabling researchers to train and test models for cancer detection and biomarker identification. It is high-dimensional and class-imbalanced, making it suitable for evaluating feature selection, dimensionality reduction, and lightweight DNN schemes based on KD, pruning, and quantization-aware training. Biologically, it captures the main patterns of miRNA dysregulation, a crucial factor in tumor initiation and metastasis. It also provides a reproducible benchmark for computational and biomedical studies of miRNA-based BCD and AI research. The dataset is freely and publicly available under the CC0 Public Domain license on Kaggle: <https://www.kaggle.com/datasets/rhostam/primary-breast-cancer-vs-normal-breast-tissue> (Kaggle PBCNT Dataset) (accessed on 2 November 2025) [58].

3.4.2. Wisconsin Breast Cancer Database (BCDD)

The BCDD dataset was collected at the University of Wisconsin Hospitals in Madison and assembled by Dr. William H. Wolberg [59]. The abbreviation used in this paper is

BCDD in order to distinguish its acronym from the term BCD (Breast Cancer Detection) and the dataset WDBC (Breast Cancer Wisconsin (Diagnostic) Dataset). It is a reputable dataset in the study and diagnostic modeling of breast cancer. All its records are cytological observations from Fine-Needle Aspirates (FNAs) of breast masses. FNA is a diagnostic procedure that uses a fine needle to collect cellular material from suspicious breast lesions. The data contains 699 instances (samples), each represented by a sample ID, and ten numerical features that define cell characteristics of the breast tissue. These are clump thickness, cell size and shape uniformity, marginal adhesion, single-cell size of the epithelial cell, bare nuclei, bland chromatin, normal nucleoli, and mitoses, each rated out of 10.

The attributes represent morphological changes of cell nuclei between benign and malignant tumors. The last column of the dataset is the class label, which can take two values: benign or malignant. The distribution of the classes is relatively unbalanced, having 458 benign samples (65.5%) and 241 malignant samples (34.5%) [60]. There are only 16 cases of missing attribute values, denoted by a question mark. It has a clean, well-documented structure, ensuring the dataset can be used for statistical analysis, classification, and feature selection in biomedical ML experiments. The data is publicly available on Kaggle under the CC0 Public Domain license: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data> (Kaggle BCDD Dataset) (accessed on 2 November 2025) [60], which grants open-access and reproducibility to scholars.

3.4.3. Breast Cancer Wisconsin (Diagnostic) Dataset (WDBC)

The WDBC dataset, like BCDD, is also based on FNA images of breast masses [61], providing a numeric description of these extracted cells. However, BCDD describes cellular cytology (what cells appear to look like under the microscope), whereas WDBC measures cellular morphology (how cells' shapes and textures are quantified algorithmically) in relation to benign and malignant tumors. The instances represent samples of morphological and textural measurements of the cell nuclei. It consists of 569 cases—357 benign (62.74%) and 212 malignant (37.26%)—with no missing data.

The dataset contains 10 fundamental features calculated from the nuclei in the FNA images: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension. All these base attributes are further represented by their averages, standard errors, and worst (maximum) values. Hence, 30 numerical features are generated per sample. The dataset is a benchmark that combines size-, shape-, and texture-related information, making it suitable for building advanced ML models to support early BCD. It provides insights into feature extraction, dimensionality reduction, classification, and the detection of possible morphological biomarkers associated with malignancy. It is accessible on the Kaggle site under the CC0 license: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data> (Kaggle WDBC Dataset) (accessed on 2 November 2025) [62].

4. Proposed System

This section presents our proposed model, describing the models we developed, trained, and tested. Figure 3 explains the processes. It starts with obtaining and preprocessing the dataset (i.e., PBCNT, BCDD, or WDBC). The preprocessing stage allows us to inspect for missing values and non-informative columns (e.g., id) and remove them. Then the features are separated from the class columns, because they require different preprocessing techniques. The class is then mapped to binary values ($B = 0$, $M = 1$, where B and M represent Benign and Malignant, respectively) and converted to one-hot encoding for training. It is vital to convert the categorical data to one-hot coding to enable the model to learn more accurately. The features were normalized to ensure consistent feature scaling

across the training and test splits and to support an accurate Principal Component Analysis (PCA) for dimensionality reduction. Then PCA was applied to remove features that did not add information to the data. Finally, the processed feature matrix was reshaped into a format suitable for the input layer of TM and SM.

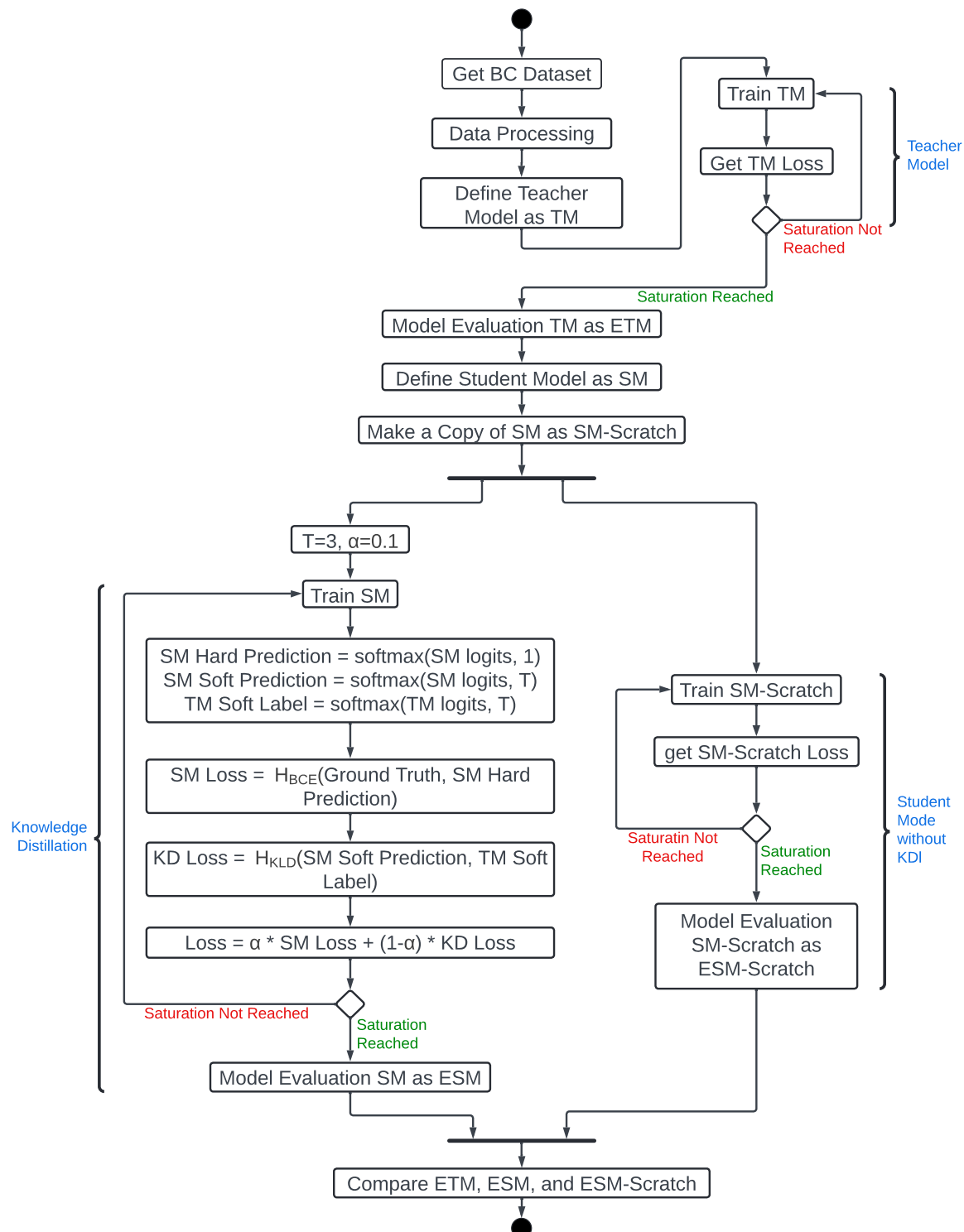


Figure 3. Experiment Carried Out.

We used the preprocessed dataset to train the TM. We also developed a lighter model with fewer parameters than the TM that can run on embedded or IoT devices. This light model is called the SM. SM was duplicated to create SM_Scratch. Then, we used the TM

to teach SM about the dataset using the KD technique. However, SM_Scratch was trained from scratch without using the KD technique, serving as a control for our experiment to assess how well SM performed.

The details of this model can be found in Table 3. It shows the model summary of TM and SM on the WDBC dataset without any dimensionality reduction. To construct an effective TM, we used the hyperband grid search technique. It identifies the optimal hyperparameters, including the number of hidden layers, filter sizes, dropout units, and learning rate.

Table 3. Teacher and student model summary.

Network		Teacher		Student
Layer	Output Shape	Parameters	Output Shape	Parameters
Conv1D	(None, 29, 64)	192	(None, 29, 4)	12
Batch	(None, 29, 64)	256	(None, 29, 4)	16
Dropout	(None, 29, 64)	0	(None, 29, 4)	0
Conv1D	(None, 28, 448)	57,792	(None, 28, 8)	72
Batch	(None, 28, 448)	1792	(None, 28, 8)	32
Dropout	(None, 28, 448)	0	(None, 28, 8)	0
Flatten	(None, 12,544)	0	(None, 224)	0
Dense	(None, 64)	802,880	(None, 2)	450
Dropout	(None, 64)	0		
Dense	(None, 2)	130		
Total Parameters:		863,042		582
Trainable Parameters:		862,018		558
Non-trainable Parameters:		1024		24

The resulting TM consists of an input layer, three hidden layers, and an output layer. The hidden layers are two Conv1D (1D convolutional) layers and a dense layer. Conv1D layers perform convolutional operations in which the kernel iterates over the feature vector and computes dot products with adjacent feature values to generate informative feature maps. To stabilize learning and enhance accuracy, batch normalization is applied after each convolution. Both Conv1D layers use ReLU activation with a kernel size of 2. The first convolutional layer has 64 filters and transforms the 30-by-1 input into a 29-by-64 tensor. The second convolutional layer uses 448 filters to process these representations into a 28-by-448 tensor further.

Dropout layers follow the convolution blocks. They randomly deactivate 10% of neurons during training, reducing overfitting and improving generalization. The model then flattens the final 28-by-448 feature map into a single vector of 12,544 elements and feeds it into two dense layers with ReLU and sigmoid activations. Therefore, the total number of parameters in the TM is 863,042, of which 862,018 are trainable, and 1024 are non-trainable.

The SM is a shallow CNN model with significantly fewer parameters than the TM. As shown in Table 3, it contains two lightweight Conv1D layers, each followed by batch normalization and dropout, and a final dense classification layer. Using the same kernel size of 2, the first Conv1D layer transforms the input into a 29-by-4 tensor, and the second expands it to 28-by-8, capturing only the essential patterns needed for classification. The output is then flattened into a 224-dimensional vector and passed to a single sigmoid-activated dense layer. Unlike the TM, the SM contains no intermediate dense layers, reflecting its intentionally shallow architecture. The SM has 582 total parameters, of which 558 are trainable, and 24 are non-trainable, corresponding to a 99.93% reduction relative to the TM while preserving strong predictive performance when trained with knowledge distillation.

To provide a clear, step-by-step representation of the proposed approach, Algorithm 1 outlines the complete workflow of our KD framework for lightweight BCD. This pseudocode complements the flowchart in Figure 3 by formalizing the operations involved in preprocessing, training the Teacher Model (TM) and Student Model (SM), and applying knowledge distillation.

Algorithm 1 Knowledge Distillation for Lightweight CNN in Breast Cancer Detection

Require: Breast cancer dataset D , Temperature $T = 3$, weight $\alpha = 0.1$

Ensure: Trained Teacher & Student Model (SM with KD) and SM-Scratch (without KD)

- 1: **Preprocessing:**
 - (a) One-hot encode categorical labels
 - (b) Use Principal Component Analysis (PCA) to remove low-information features
 - (c) Perform a stratified train-validation split to preserve class proportions
 - (d) *Optional:* Apply KMeans-SMOTE oversampling to the training dataset in order to achieve class balance
- 2: **Define Teacher Model (TM)**
- 3: **Train TM** on processed D until convergence
- 4: **Evaluate TM:** Accuracy, AUC, Precision, Recall
- 5: **Define Student Model (SM)**
- 6: **Train SM-Scratch (without KD):**
- 7: **while** not converged **do**
- 8: Train SM-Scratch using ground truth labels only
- 9: **end while**
- 10: Evaluate SM-Scratch: Accuracy, AUC, Precision, Recall
- 11: **Train SM with Knowledge Distillation:**
- 12: **while** not converged **do**
- 13: *Forward pass:*
- 14: $q'_{SM} \leftarrow \text{softmax}(\text{SM logits}, \tau = 1)$ {hard prediction}
- 15: $q_{SM} \leftarrow \text{softmax}(\text{SM logits}, \tau = T)$ {soft prediction}
- 16: $p_{TM} \leftarrow \text{softmax}(\text{TM logits}, \tau = T)$ {teacher soft label}
- 17: *Compute losses:*
- 18: SM Loss $\leftarrow H_{BCE}(y, q'_{SM})$ { H_{BCE} : Binary Cross Entropy}
- 19: KD Loss $\leftarrow H_{KLD}(q_{SM}, p_{TM})$ { H_{KLD} : KL Divergence}
- 20: Total Loss $\leftarrow \alpha \times \text{SM Loss} + (1 - \alpha) \times \text{KD Loss}$
- 21: Backpropagate and update SM parameters
- 22: **end while**
- 23: Evaluate SM: Accuracy, AUC, Precision, Recall
- 24: **Compare** TM, SM, and SM-Scratch performances
- 25: **Output** final SM (with KD) for deployment

5. Materials and Methods

This section describes the experiments used to obtain the results in Section 6. All the experiments were carried out using the procedures in this section; the slight variations are discussed in the respective subsections of Section 6.

The initial experiment investigates the diagnostic performance of the proposed system, which was assessed using three primary evaluation metrics: accuracy, precision, and recall, computed according to Equations (3)–(5). They measure the system's capability to correctly identify malignant and benign breast tissue samples, assess the reliability of its positive predictions, and determine its sensitivity in detecting cancerous cases. The deployability test measures the file size, inference time, percentage CPU and memory usage, CPU temperature, and power consumption on an SBC. Raspberry Pi 4 Model B was the SBC used in the experiments. Additionally, the section compares the proposed model with the PTQ and QAT techniques in terms of accuracy, file size, number of parameters, inference

time, CPU and memory usage percentages, CPU temperature, and power consumption. The codes for these experiments are publicly available on GitHub [63].

$$Accuracy = \frac{TP + TN}{TP + TN + FN + FP} \quad (3)$$

$$Precision = \frac{TP}{TP + FP} \quad (4)$$

$$Recall = \frac{TP}{TP + FN} \quad (5)$$

where

TP = number of malignant cases correctly classified as malignant,

FP = number of benign cases misclassified as malignant,

TN = number of benign cases correctly classified as benign,

FN = number of malignant cases misclassified as benign.

The deployability performance was evaluated by analyzing inference latency, power, CPU, and memory usage on a RPi Model 4 B. Figure 4 shows the experimental setup for measuring power consumption on the RPi. It features a RPi Model 4B as the embedded computing system. It represents IoT devices that could host the proposed model. We connected to the RPi from a second computer via SSH over Wi-Fi. A DC-DC buck converter powers the RPi. We connected the buck converter, the ACS712-5B current sensor input, and a 12 V DC power supply in series to measure the load's current consumption (i.e., the RPi). This connection allows the current sensor to measure the current (I) drawn by the RPi from the power supply during the experiment and send it to an Arduino Nano for analysis. The Arduino Nano also measures the voltage drop (V) across the RPi. The Arduino sends the measurement through USB to a laptop for collation. We used Equation (6) to measure the power (P) consumption of the RPi. The power consumption measurements were averaged across five experiments to reduce noise.

$$P = V \times I \quad (6)$$

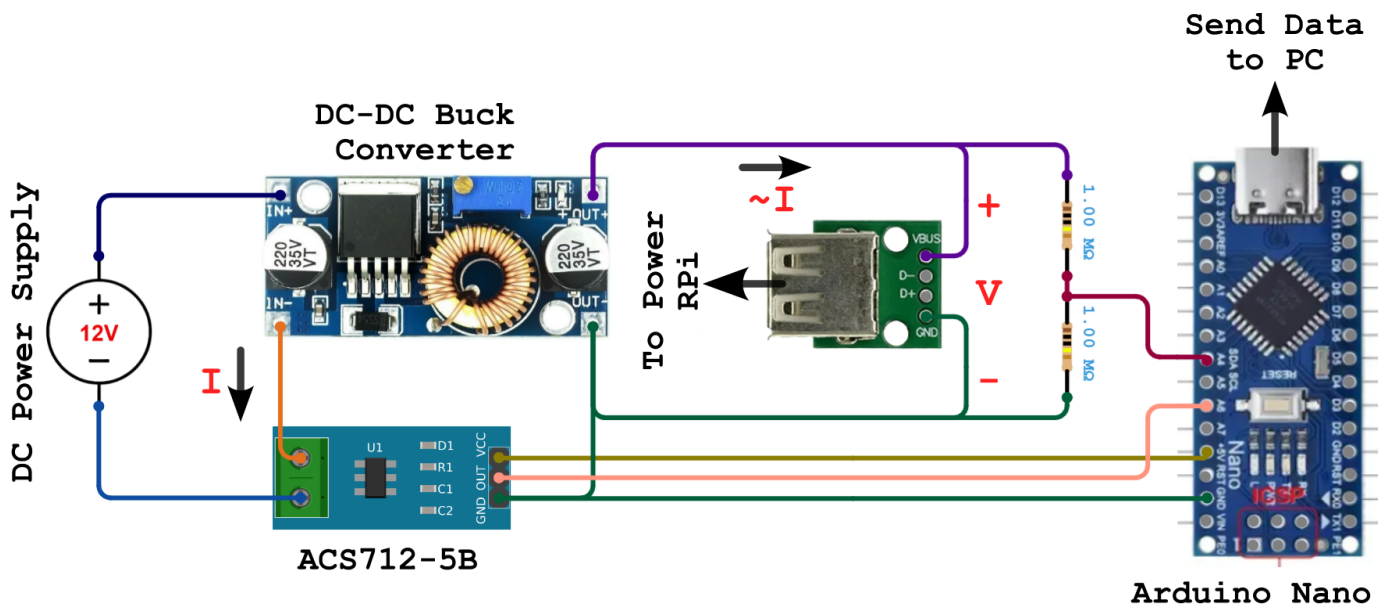


Figure 4. Deployability Experimental Setup.

For the latency measurement, we run the code in Algorithm 2. It loads the data and the model. Then it typecasts the data and tests a single data instance against the model. We used Python's Time v0.0.17 package to record the time immediately before and after

loading the model (t_{LM}), loading the input and output tensors (t_{LT}), data type casting (t_{prep}), and inference (t_i) and subtracting the before and after time as shown in Equations (7), (8), (9), and (10), respectively. We noticed low latency variation, so no average was taken. Rather, the value of the fifth iteration was recorded. Additionally, all model evaluations on the RPi 4 Model B were performed in CPU-only mode under the default RPi OS (Raspbian 11) installation. The Python scripts were executed directly without any custom compilation or backend reconfiguration. The TFLite v2.18.0 automatically employed the XNNPACK CPU delegate, which internally utilizes the ARM NEON instruction set for optimized matrix operations. No GPU, OpenCL, NNAPI, or TensorRT delegates were enabled, ensuring that all inference was executed solely on the CPU.

Algorithm 2 Inference Evaluation Experiment

Require: Model File, Pre-Processed Dataset, Time v0.0.17

```

1:  $X \leftarrow$  Load feature of Pre-Processed Dataset
2:  $Y \leftarrow$  Load feature of Pre-Processed Dataset
3:  $t_{LM,1} \leftarrow$  time now
4: Model  $\leftarrow$  Load Model File
5:  $t_{LM,2} \leftarrow$  time now
6:  $t_{LT,1} \leftarrow$  time now
7:  $T_{input} \leftarrow$  get input tensor of Model
8:  $T_{output} \leftarrow$  get output tensor of Model
9:  $t_{LT,2} \leftarrow$  time now
10:  $t_{prep,1} \leftarrow$  time now
11:  $X_{prep} \leftarrow$  datatype of  $T_{input}$ 
12:  $t_{prep,2} \leftarrow$  time now
13:  $t_{i,1} \leftarrow$  time now
14:  $Y_{pred} \leftarrow$  Model( $X_{prep}[0]$ )
15:  $t_{i,2} \leftarrow$  time now

```

$$\Delta t_{LM} = t_{LM,2} - t_{LM,1} \quad (7)$$

$$\Delta t_{LT} = t_{LT,2} - t_{LT,1} \quad (8)$$

$$\Delta t_{prep} = t_{prep,2} - t_{prep,1} \quad (9)$$

$$\Delta t_i = t_{i,2} - t_{i,1} \quad (10)$$

Regarding the CPU and memory usage, a Python-based monitoring script was developed to record the Raspberry Pi's performance metrics, including CPU utilization, memory usage, and CPU temperature. The script periodically sampled the device's hardware statistics and stored them in a structured format at one-second intervals. It runs on an indefinitely loop until manually stopped. The monitoring script was executed in a manner that redirected its output to a log file, allowing the performance measurements to be continuously recorded. This experiment was also averaged over five iterations to reduce noise from background processes.

6. Results and Discussion

This section presents and discusses the performance of the proposed model. It investigates its diagnostic and deployability performances. It is organized into five subsections for clarity and completeness. Section 6.1 presents an ablation study on the KD hyperparameters to investigate their influence on model performance. Section 6.2 examines the efficacy of KD as an extension of our previous work [11], evaluating performance across folds in a 10-fold cross-validation. It also benchmarks the SM against related studies, assesses its deployability on the RPi, and evaluates its cross-dataset performance under domain shift. Section 6.3 investigates the benefits of optimizing the model using the TFLite framework

in terms of accuracy, file size, architectural compactness, and deployability on embedded devices such as the RPi 4 Model B. Sections 6.4–6.6 analyze the performance of the pruning technique, PTQ, and QAT, respectively, focusing on their impact on accuracy, compression, and deployability on embedded systems, and comparing their results with the TFLite version of SM.

6.1. Ablation Study on Distillation Hyperparameters

This section presents an ablation study to investigate optimal values for the KD hyperparameters: the distillation coefficient (α) and the temperature (T). The coefficient α controls the trade-off between the hard label loss (H_{BCE}) and the soft label loss (H_{KLD}), while T determines the smoothness of the teacher's output probabilities [47]. Thus, it is necessary to determine the optimal combination of the two hyperparameter values. The experiment uses $\alpha \in \{0.1, 0.3, 0.5\}$ and $T \in \{2, 3, 5, 7, 10\}$. We selected lower values for T because researchers posited that lower T values tend to produce sharper probability distributions, which can benefit smaller student models [47]. The results of the experiments are visualized as a heatmap in Figure 5.

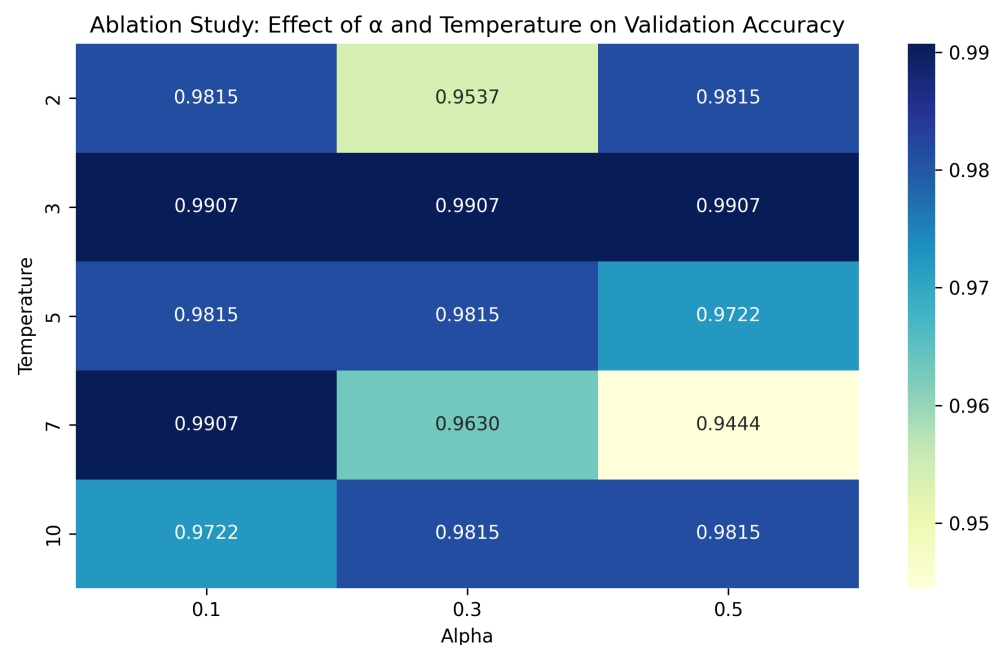


Figure 5. Heatmap of validation accuracy (%) for different α and T values in the distillation process.

The heatmap in Figure 5 shows the validation accuracy of proposed models for different values of α and T . It shows that the highest validation accuracy is 99.07% and it was achieved when $T = 3$ across all values of α (i.e., 0.1, 0.3, or 0.5). These results show that $T = 3$ consistently delivers top performance for all α values and that it is the optimal value that strikes a balance between preserving informative soft targets and avoiding overly smooth probability distributions. Regarding the α values, we can see that the highest validation accuracy was recorded at $\alpha = 0.1$ and $T = 7$. Additionally, the $\alpha = 0.1$ column shows that it consistently outperforms other combinations, except for only one case ($T = 10$). Hence, lower α values (e.g., 0.1) produced more stable performance. This result indicates that placing too much emphasis on the hard label loss can diminish the benefits of distillation when the teacher's outputs are overly smoothed. Therefore, we selected $\alpha = 0.1$ and $T = 3$ for all KD experiments in this research.

6.2. Performance Evaluation of KD

This section evaluates the proposed model (SM) by comparing it with the TM and SM_Scratch. It also compares the proposed SM with existing literature, using three key performance metrics: accuracy, precision, and recall. The experiments evaluated the models across three benchmark datasets: WDBC, PBCNT, and BCDD. Evaluating performance across multiple datasets provides a broader assessment of the model's robustness and verifies its predictive consistency across different data distributions. Finally, this section presents a cross-dataset analysis in which the models are trained on WDBC and tested on BCDD to examine their behavior under distributional shift.

6.2.1. Diagnostic Performance of KD

This section uses 10-fold cross-validation on the WDBC dataset to provide an excellent balance between bias and variance in the models' performance estimate. Figure 6 presents the results of the experiments over 30 epochs. It shows the training and validation accuracy and loss for the TM, SM_Scratch, and SM. The comparison highlights the performance differences among the three models during both training and validation phases. The error bars represent 95% Confidence Intervals (CIs) for the mean values across the ten folds, reflecting the stability and consistency of each model. This visualization provides an overview of how effectively KD improves the student model's learning behavior compared to the teacher and the standalone student network.

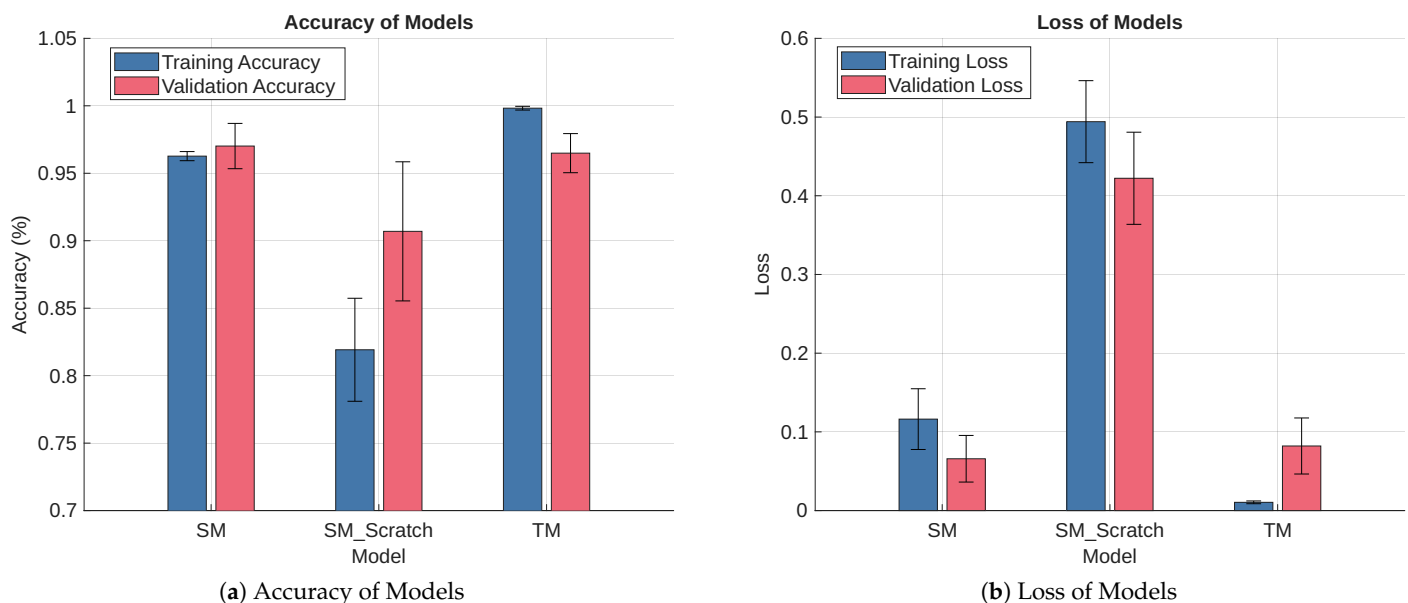


Figure 6. Diagnostic Performance of TM, SM_Scratch, and SM in a 10-Fold Cross-Validation Setting.

Figure 6a shows the training and validation accuracies of the 10-fold cross-validation experiment. The results indicate that TM achieved the highest training and validation accuracies of $99.82\% \pm 0.14\%$ and $96.49\% \pm 1.45\%$, respectively. Notice that the CIs are small, indicating strong convergence and generalization. This outcome is intuitive because TM has a larger network capacity than SM_Scratch and SM, allowing it to capture complex, subtle, and high-dimensional patterns. SM has the second-best performance. It reaches $96.27\% \pm 0.34\%$ and $97.02\% \pm 1.68\%$ for training and validation accuracy, respectively. The CIs are also small, indicating that the SM converges and generalizes well.

Additionally, the training and validation accuracy CIs of SM overlap the validation accuracy CI of TM, indicating that their performance is statistically comparable at the 95% confidence level. However, SM_Scratch performed significantly worse, with a training

accuracy of $81.92\% \pm 3.82\%$ and validation accuracy of $90.69\% \pm 5.16\%$. The wide CIs for both accuracies in SM_Scratch reflect high fold-to-fold variability, slower learning, and weaker generalization. The partial overlap of its validation accuracy CI with those of TM and SM (and its training accuracy) suggests that the observed differences in mean accuracy may not be statistically significant, given the limited sample size. Comparing SM and SM_Scratch shows that SM effectively learned from the TM.

Figure 6b shows the loss results of the models. The loss evaluation mirrors the accuracy trends. TM recorded the lowest training loss of 0.010 ± 0.002 and the validation loss of 0.082 ± 0.036 , confirming its well-fitted learning behavior. SM achieved a slightly higher training loss of 0.116 ± 0.039 but a lower validation loss of 0.066 ± 0.030 , suggesting better generalization to unseen data. Similar to the accuracy results, the training and validation CIs of SM overlap, and both overlap the validation loss CI of TM, reinforcing the conclusion that the performance of TM and SM is statistically comparable at the 95% confidence level. SM_Scratch again showed the weakest performance, with high losses of 0.494 ± 0.052 for training and 0.422 ± 0.059 for validation, consistent with underfitting. Its CIs are well above those of TM and SM, supporting the interpretation that its apparent overlap in accuracy is likely due to limited sample size rather than comparable performance. In general, these results demonstrate that KD stabilizes training, improves generalization, and enables the student model to achieve teacher-level accuracy with substantially reduced complexity.

6.2.2. Benchmarking Against Related Work

In this section, the SM was trained without cross-validation, using 85% of each preprocessed dataset for training and the remaining 15% for validation, with 30 training epochs. KMeans-SMOTE was applied to ensure class balance while preventing data leakage. Although six experiments were initially conducted (with and without oversampling for each dataset), we report results only for configurations that yielded unique outcomes. Specifically, oversampling did not affect the WDBC and PBCNT datasets. The oversampled and baseline models achieved identical performance. So we report only the baseline results for these datasets. For the BCDD dataset, oversampling eliminated a single false negative observed in the baseline model; therefore, both SM-BCDD and SM-BCDD-O are included. Table 4 summarizes the experiment nomenclature.

Table 4. Nomenclature for the Experiments Reported.

Experiments	Description
SM-WDBC	WDBC Dataset (baseline)
SM-PBCNT	PBCNT Dataset (baseline)
SM-BCDD	BCDD Dataset (baseline)
SM-BCDD-O	BCDD Dataset with oversampling

Table 5 summarizes the performance of the proposed KD-based student models alongside recent related work using the same numerical breast cancer datasets. Oversampling did not produce a measurable change in the WDBC and PBCNT datasets, indicating that they are either naturally balanced or sufficiently separable. For the BCDD dataset, oversampling removed the single false negative observed in the baseline configuration, resulting in a modest improvement in recall. Across all experiments, the student model achieved very low false-positive and false-negative counts (≤ 1), demonstrating high diagnostic reliability and sensitivity.

Table 5. Performance comparison with related works.

Sn	Ref	Dataset	Algorithm	Acc. (%)	Prec. (%)	Recall (%)	F1-Score
1	SM-WDBC	[62]	KD-CNN	98.8	100.0	97.0	0.98
	SM-PBCNT	[58]	KD-CNN	100.0	100.0	100.0	1.00
	SM-BCDD	[60]	KD-CNN	98.1	97.3	97.3	0.97
	SM-BCDD-O	[60]	KD-CNN	99.0	97.3	100.0	0.99
2	[64]	[60]	DWT-ANN	98.8	98.3	98.3	0.98
3	[65]	[62]	SVM	97.2	99.5	94.8	0.97
			RF	96.5	96.6	92.5	0.95
			LR	95.8	97.6	95.0	0.96
			DT	95.1	94.0	92.0	0.93
			KNN	93.7	96.6	94.8	0.96
4	[66]	[62]	ANN	97.0	100.0	94.0	97.0
5	[67]	[60]	SVM	97.7	99.0	96.0	0.97
			ANN	97.1	100.0	93.0	0.96
			XGBoost	95.9	97.0	93.0	0.95
			RF	95.3	97.0	91.0	0.94
			DT	94.6	95.0	87.0	0.91
6	[68]	[62]	SVM	98.5	98.0	99.0	0.98
			RF	98.2	98.4	98.0	0.98
			DT	97.0	97.0	97.0	0.97
			LR	96.8	97.6	96.0	0.97
			KNN	96.0	96.0	96.0	0.96
7	[69]	[62]	CNN	87.9	97.9	84.8	0.91

Qualitative Error Analysis and Feature-Level Explainability

This section provides a qualitative analysis of the FPs and FNs observed across the four SM configurations in Table 4. Confusion matrices are first used to characterize each model's classification behavior and to identify any misclassified samples. To better understand the factors contributing to these errors and to assess whether the SM relied on clinically meaningful patterns, we complemented this analysis with feature-level attribution using Shapley Additive exPlanations (SHAP). Figure 7 presents the confusion matrices for the four models, while Figure 8 shows the normalized SHAP contributions of the features that most strongly influenced the misclassifications.

Figure 7a presents the confusion matrix for SM-WDBC. It shows one FN, where a malignant case was misclassified as benign, and no FPs. To investigate which features contributed to this misclassification, Figure 8a displays the SHAP values computed for the misclassified sample. The plot indicates that *area_worst* and *area_mean* were the most influential features in misleading the model, with additional contributions from *smoothness_worst* and *symmetry_mean*. These features collectively reduced the predicted malignancy probability and pushed the model's decision toward the benign class. Thus, the FN corresponds to a borderline case with only mildly abnormal morphology, an ambiguity that is challenging even for experienced cytopathologists.

Unlike the other configurations analyzed in this section, SM-PBCNT produced no misclassifications, as shown in Figure 7b. Because this is the only case without FPs or FNs, we examined borderline correct predictions by computing SHAP values for the samples with the lowest predicted probabilities. Figure 8b presents the resulting SHAP values for these least-confident correctly classified instances. The feature-level analysis revealed that *hsa-miR-2964a-5p*, *hsa-miR-3613-3p*, *hsa-miR-4668-5p*, *hsa-miR-125b-5p*, *hsa-miR-451a*, and *hsa-miR-21-5p* were the most influential contributors, helping the model correctly

classify these borderline cases. These miRNAs also aligned strongly with the principal components identified through PCA, indicating that SM-PBCNT captured biologically meaningful expression patterns rather than overfitting.

Figure 7c shows that the SM-BCDD model produced one FP and one FN, resulting in two separate SHAP explanations. For the FP case, the misclassification was driven primarily by high values of *marginal_adhesion* and *bare_nucleoli*. For the FN case, the misclassification was influenced mainly by high *marginal_adhesion* and *shape_uniformity*, causing the malignant sample to exhibit benign-like patterns and thereby suppressing its predicted malignancy score. Notably, *marginal_adhesion* was the strongest contributor in both cases. Figure 8c presents the mean SHAP values across the FP and FN cases, providing a unified view of the features contributing to misclassification. The aggregated contributions highlight *marginal_adhesion*, *bare_nucleoli*, and *shape_uniformity* as the three most influential features.

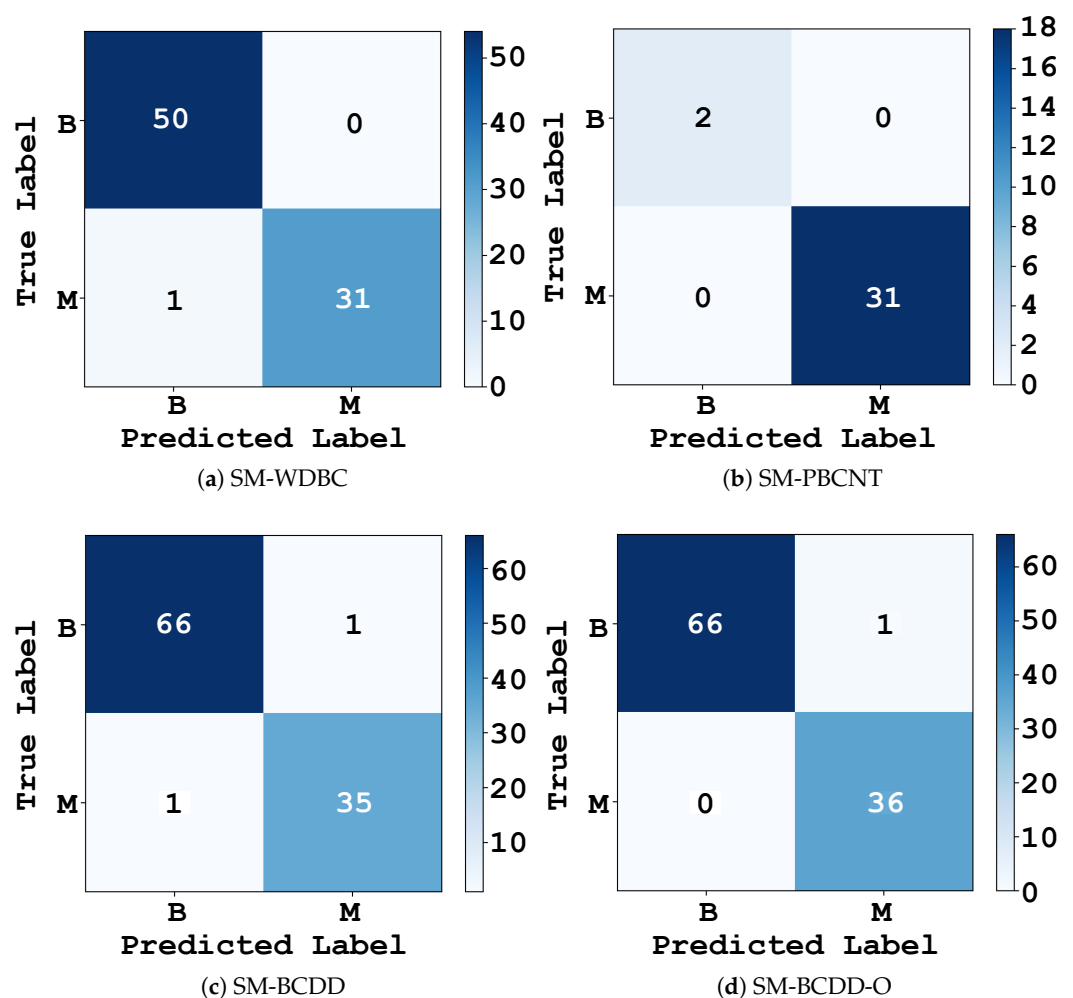


Figure 7. Confusion Matrices for SM Across the Four Evaluated Datasets.

For the SM-BCDD-O experiment, a single FP was observed as shown in Figure 7d. The SHAP values in Figure 8d indicated that *clump_thickness* was the dominant contributor, accounting for nearly half of the total attribution, followed by *shape_uniformity*, *size_uniformity*, *bare_nucleoli*, and *epithelial_size*. This benign sample exhibited unusually high epithelial clustering and nuclear changes, making it appear more similar to malignant cases and triggering an overly sensitive malignant prediction. Compared to SM-BCDD, the oversampled model placed greater emphasis on clump thickness and shape-related features, trading a slight increase in FPs for the complete removal of FNs.

Therefore, across all four datasets, the qualitative error analysis and SHAP-based feature attributions confirm that the distilled SM relies on domain-consistent miRNA expression features in PBCNT and well-known morphological descriptors in WDBC and BCDD. The remaining errors are mainly found in cytologically borderline cases.

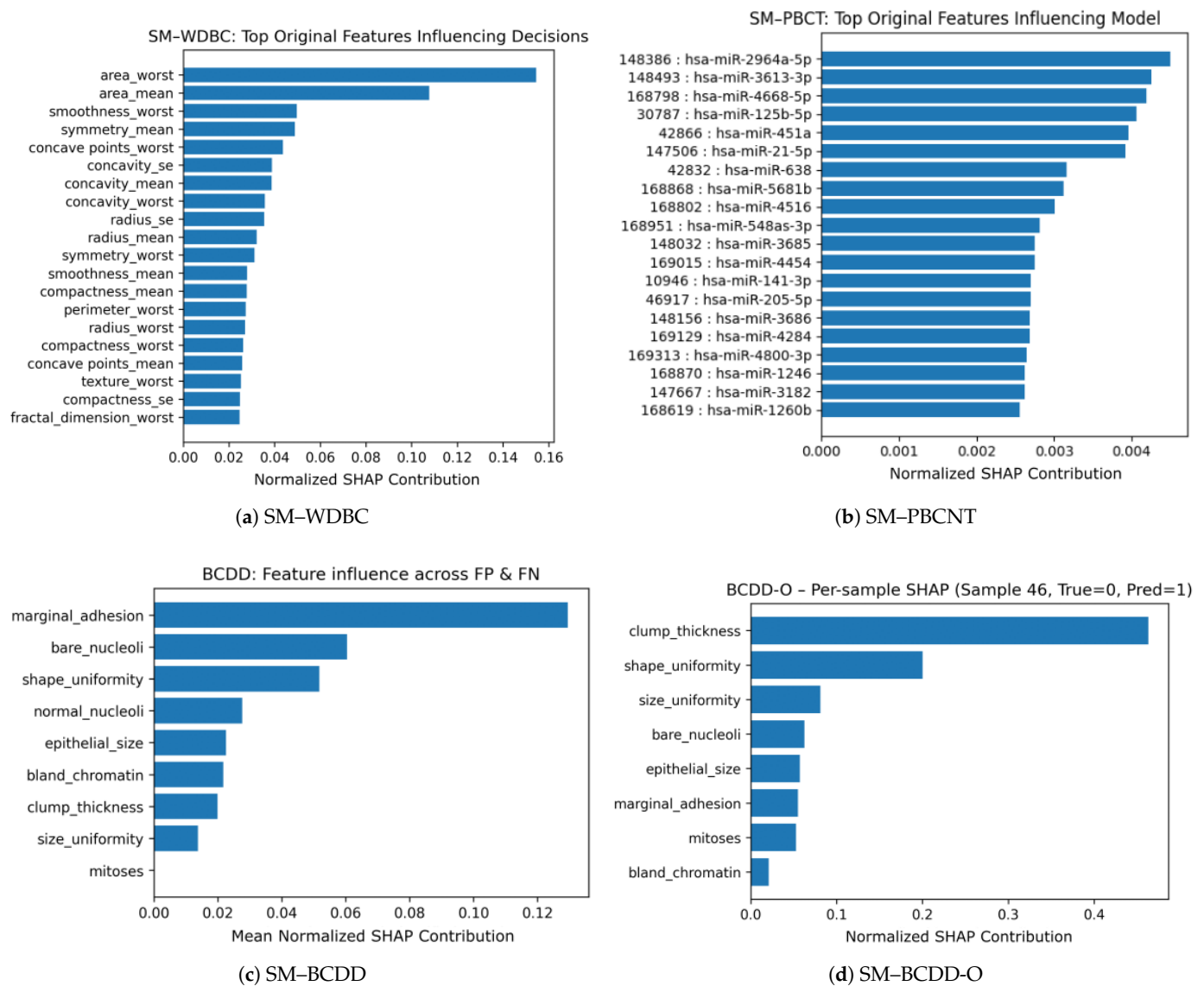


Figure 8. Feature-level SHAP Importance for the Four Reported Experiments.

Operating Points

To further evaluate diagnostic reliability, we assessed both discrimination and probabilistic calibration using the predicted probability of the malignant class. The ROC curve was generated by comparing the predicted malignant probability against the ground-truth labels, and the area under the curve (AUC) was computed from the obtained set of False-Positive Rates (FPRs) and True-Positive Rates (TPRs). As shown in Figure 9, the SM demonstrates excellent separability between benign and malignant samples, with $0.997 \leq \text{AUC} \leq 1.000$ across all datasets.

Because screening applications prioritize sensitivity, we selected clinically meaningful operating thresholds by enforcing a minimum recall of 95%. This was implemented by scanning the whole ROC curve and extracting all thresholds where $\text{TPR} \geq 0.95$.

From this subset, we selected the threshold with the lowest FPR, thereby maximizing specificity while preserving the required sensitivity. Table 6 reports the optimal thresholds and corresponding performance values. The SM consistently achieved $\geq 96\%$ sensitivity while maintaining extremely low FPRs (≤ 0.0185), demonstrating suitability for real-world screening workflows.

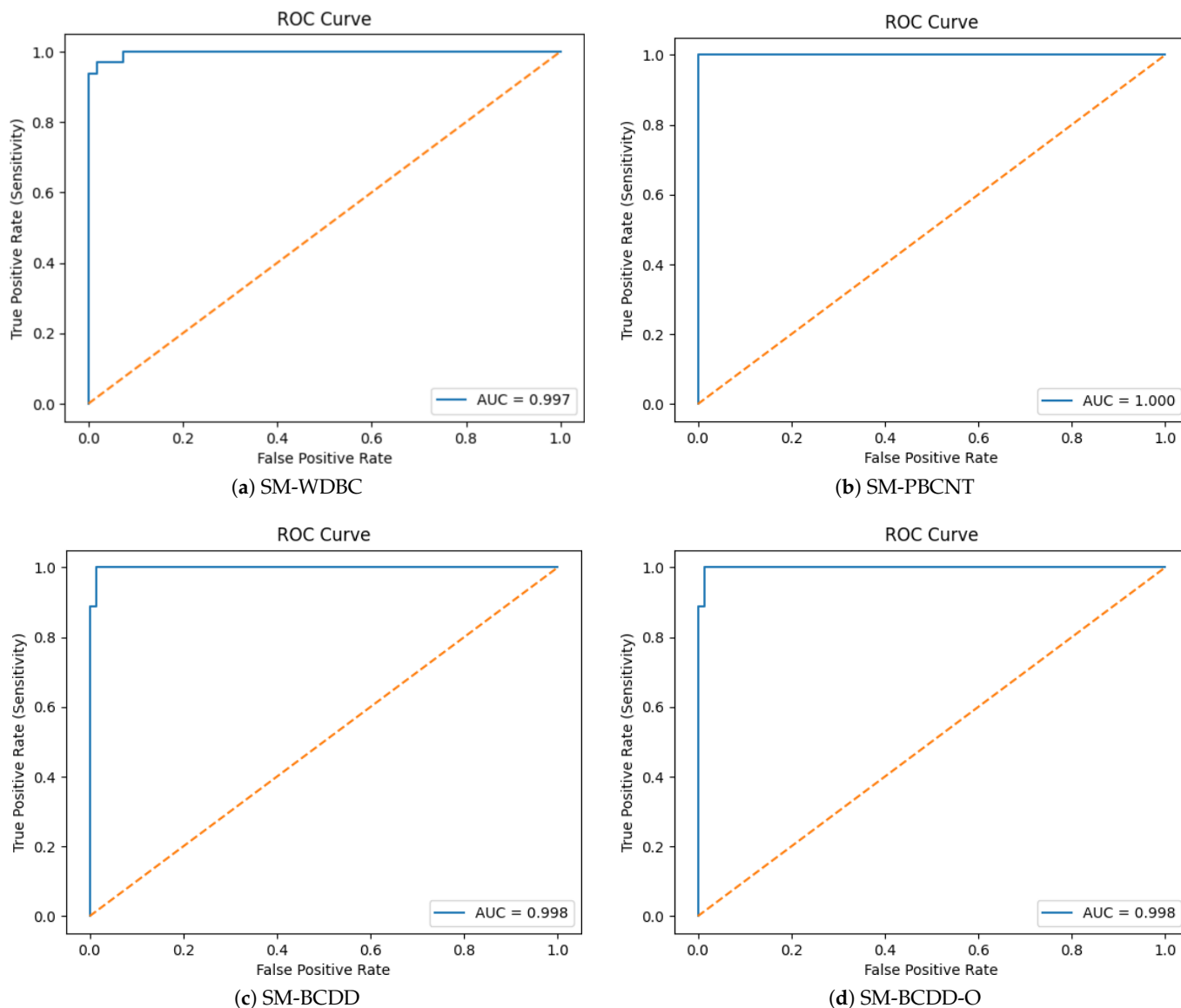


Figure 9. ROC Curves.

Table 6. Thresholds and Performance Metrics at $\geq 95\%$ sensitivity.

Dataset	Threshold	Sensitivity	Specificity	FPR
SM-WDBC	0.4280	0.9688	0.9815	0.0185
SM-PBCNT	0.6072	1.0000	1.0000	0.0000
SM-BCDD	0.5840	1.0000	0.9851	0.0149
SM-BCDD-O	0.2876	1.0000	0.9851	0.0149

Calibration Analysis

Calibration was evaluated using reliability (calibration) curves, which compare the model's predicted probabilities with the observed frequency of positive cases. To construct these curves, we divided the predicted probabilities from each configuration into 10 bins

and, for each bin, computed the mean predicted probability and the empirical fraction of malignant cases. Perfect calibration corresponds to points lying on the diagonal reference line. As shown in Figure 10, all four configurations align closely with this diagonal, indicating good probability calibration. The SM-BCD-O configuration exhibits the best alignment, with predicted probabilities closely matching true outcome frequencies; its slight upward deviation in the mid-probability range indicates minor underconfidence.

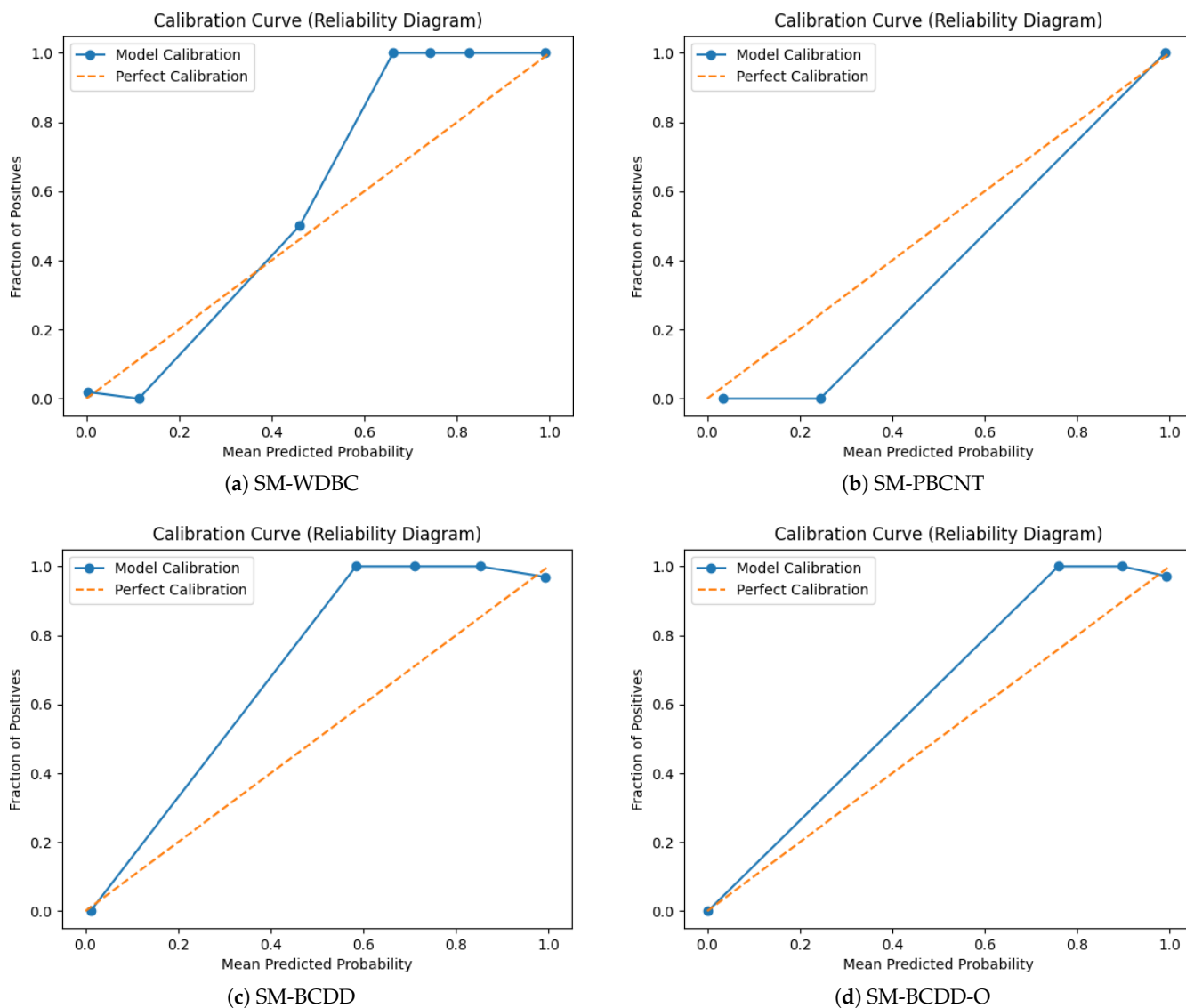


Figure 10. Calibration Curves.

6.2.3. Deployability Test for KD

This section studies the resource usage of the proposed model. Table 7 summarizes the resource usage results of the experiment on the RPi from our previous work [11], where we tested the TM, SM_Scratch, and SM. The first column contains the measured metrics, and the subsequent columns show the results for the TM, SM_Scratch, and SM. The results show that the proposed SM trains 2.7 times faster than the TM and 1.8 times slower than the SM_Scratch. However, the proposed model has a data preprocessing and inference time of 4.2 ms and 500 ms, respectively. These results are acceptable for non-real-time telemedicine applications. During training, power consumption increased to $3.0\text{--}4.5\times$ the nominal device power, while CPU utilization peaked at 88% and memory usage remained

below 12% of available RAM. The CPU temperature reached approximately 60 °C without active cooling but remained stable during inference.

Table 7. Resource Experiment Summary from Previous Work.

Metric	TM	SM_Scratch	SM
Training Latency (s)	221.68	45.01	80.74
Preprocessing Time (ms)	–	–	4.2
Inference Latency (ms)	–	–	500
Power Consumption	3–4.5× nominal during training	Similar pattern	3–4.5× nominal; +0.5 W rise during preprocessing; negligible inference overhead
CPU Usage (%)	Up to 88 (training)	Slightly lower	Up to 88 (training); ≈25 (inference)
Memory Usage(% of 8 GB RAM)	≤12	≤12	≤12
CPU Temperature (°C)	≈60 peak (no fan)	≈60 peak	≈60 peak (training); stable during inference

6.2.4. Cross-Dataset Evaluation

This section evaluates how well the models generalize when trained on one dataset and tested on another. It examines how the SM behaves under substantial distributional and feature-mismatch shift. Here, the models TM, SM_Scratch, and SM were trained on WDBC and tested on BCDD. We follow the process in Figure 11 for each model: both WDBC and BCDD were pre-processed independently, including the removal of non-informative identifiers (e.g., id) and normalization. Since the two datasets contain different numbers and types of morphological features, we applied PCA separately to each dataset and reduced them to the same number of components C . This stage enables a consistent input size for all models without imposing an artificial shared feature basis. Then, WDBC was split 70–30% for training–validation partition to allow us to investigate whether the considerable feature reduction has degraded the models’ performance. Finally, the trained models were then evaluated on the PCA-reduced BCDD dataset.

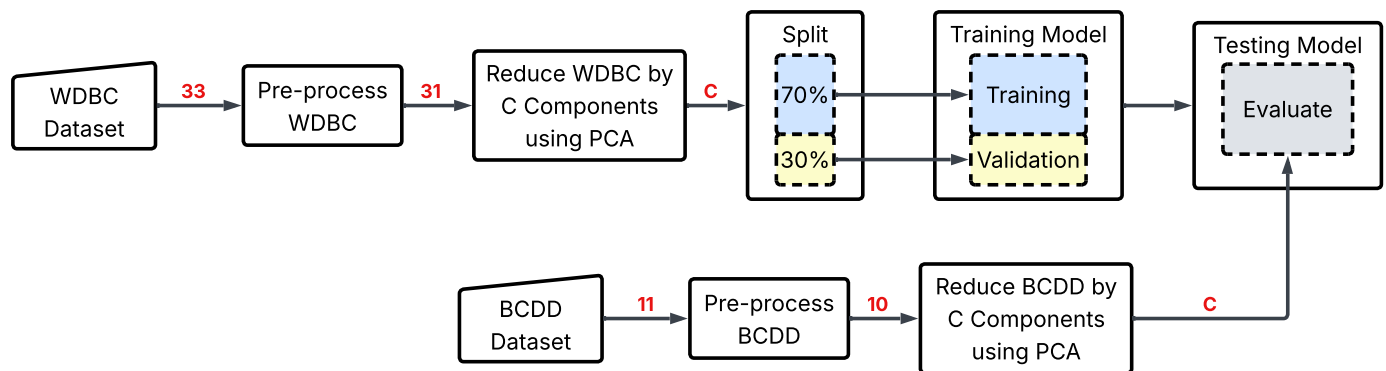


Figure 11. Cross-Dataset Evaluation Experiment.

Figure 12 summarizes the cross-dataset experiment at a 95% confidence level over ten repetitions. The training accuracy of the three models is shown in Figure 12a. It shows the TM accuracy is consistently higher than the other models, albeit slightly falling (from 96.23% to 93.12%) with the value of C . The SM_Scratch model performs the worst. It also has lower accuracy and a wider CI due to higher trial-to-trial variation. However, the accuracy of SM shows slight variation across C . It also outperforms SM_Scratch in all cases, showing that it is learning from the TM.

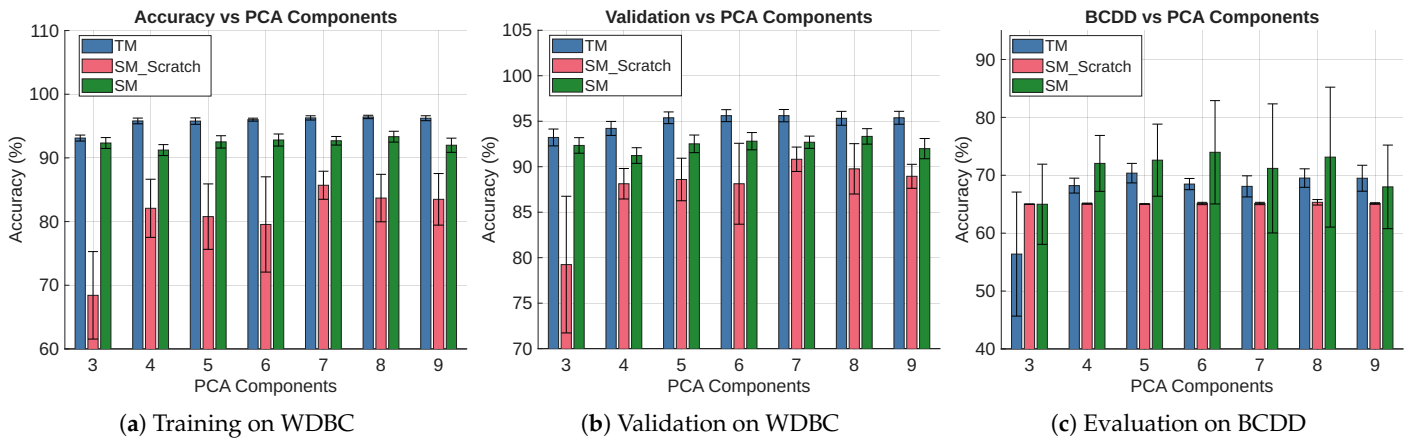


Figure 12. Performance of TM, SM_Scratch, and SM when Trained on WDBC and Tested on BCDD.

Figure 12b shows the validation results of the experiment. The results follow the same trend as the training accuracy. It shows that the TM validation accuracy closely mirrors its training accuracy. This trend is also true for the SM and SM_Scratch, suggesting that PCA reduction does not cause appreciable overfitting or divergence between training and validation performance. The result also shows that the SM outperforms SM_Scratch at $C = 9$ because it has enough features to perform better. However, there is no statistical difference between the two models for $6 \leq C \leq 8$, because the CI of the two overlap in each case. However, as the models are starved of features ($C < 6$), the SM performs better again, likely because the distillation process provides additional informative structure that compensates for the reduced feature space.

Figure 12c depicts the performance of the models when tested on the BCDD dataset. The results show that all models perform poorly on the test dataset. SM_Scratch performs the worst, consistently achieving 65.00–65.33% accuracy with slight variation across trials, as shown by the small CIs. It indicates that the model collapses toward majority-class predictions (i.e., Neural Collapse [70]), reflecting poor transferability across distinct feature domains. The TM accuracy (68.08–70.37%) is statistically the same for $4 \leq C \leq 9$. However, it performs the poorest at $C = 3$ with a wide CI (± 10.72), because it removes too much discriminative information for the TM to generalize effectively. The SM shows a wide CI throughout the experiment. Although its mean accuracy (64.99–73.97%) outperforms the TM across all PCA dimensions, they are statistically indistinguishable because their CIs overlap in every case. For the same reason, we are sure that SM outperforms SM_Scratch at $C = 4$ and $C = 5$. The figure also shows SM collapsing towards the majority class at $C = 3$, behaving like SM_Scratch rather than the TM, indicating that the feature representation at this dimensionality is insufficient for effective knowledge transfer.

6.3. TM Versus TFLite of TM, SM_Scratch, and SM

In this experiment, we converted the SM into its TFLite version (SM_TFLite). The experiment employed the same TM and SM_Scratch architectures described in Table 3. Following the procedure in Figure 3. However, after training, all three models were saved in both Keras and TFLite formats. This dual-savings process enabled us to assess differences between models stored in their native Keras format and those optimized for deployment on IoT and embedded devices via TFLite conversion.

6.3.1. Accuracy and Size Performance for TFLite

The experimental results, including accuracy, parameter counts, sparsity, and file sizes, are summarized in Table 8. The results of the TFLite conversion are summarized in

Table 8. The first three rows present the accuracy, validation accuracy, and TFLite evaluation accuracy for the TM, SM_Scratch, and SM models. In this experiment, we used TensorFlow 2.18.0, but we were unable to obtain validation accuracy during training using the Distiller class. Thus, this value is omitted for the SM.

Table 8. Performance comparison of TM, SM_scratch, and SM in Keras and TFLite formats.

SNo.	Factors	TM	SM_SCRATCH	SM
1.	Acc. (%)	100.00	75.16	97.67
2.	Val. Acc. (%)	95.35	76.74	—
3.	Acc. (TFLite) (%)	95.35	76.74	97.67
4.	Keras size (kB)	8812.02	48.67	35.03
5.	TFLite size (kB)	2924.53	5.21	5.21
6.	Total Weights	748,354	518	518
7.	Zero Weights	0	0	0

The table also shows a significant reduction in file size when the model is converted to TFLite. Rows four and five compare the file sizes of the models saved in both Keras and TFLite formats, achieving reductions of 66.81%, 89.30%, and 85.13% for TM, SM_Scratch, and SM, respectively. These reductions highlight TFLite's effectiveness in reducing model storage requirements, which is valuable for IoT and embedded systems with limited memory. Rows six and seven further analyze the total and zero weights of the models. The results show that no weights were pruned or removed during optimization, confirming that the conversion to TFLite does not change the number of parameters or introduce sparsity. Therefore, TFLite conversion significantly reduces the model size while preserving the model structure.

6.3.2. Deployability Test for TFLite

The deployability test was carried out using the setup in Section 5 and the following procedure:

1. Record the system for 30 s to obtain the baseline profile of the RPi.
2. Start the system monitor (Sys.Mon.), a Python script that logs CPU temperature along with CPU and memory usage.
3. Execute the TM_TFLite inference code and wait for the remainder of the 30 s.
4. Execute the SM_Scratch_TFLite inference code and wait for the remainder of the 30 s.
5. Execute the SM_TFLite inference code and wait for the remainder of the 30 s.
6. Continue recording for 30 s with only the system monitor running in the background before closing it.
7. Finally, record for another 30 s to capture any transient responses after closing the system monitor.

To ensure a fair comparison and eliminate transient effects, 30 s intervals were inserted between each stage of the experiment. Figure 13 shows the power consumption, CPU, and memory performances of the TFLite versions of the three models on RPi. The figures were obtained by repeating the aforementioned experiment 5 times and averaging the results. We labeled the TFLite versions of TM, SM_Scratch, and SM as TM_TFLite, SM_Scratch_TFLite, and SM_TFLite, respectively. Figure 13a presents their power consumption profiles. It shows that TM_TFLite has the highest peak at approximately 2.01 W. It is followed by SM_Scratch_TFLite at 1.96 W and SM_TFLite at 1.86 W. It also shows that all models quickly returned to the baseline idle level of around 0.8 W. The total energy consumption during execution was 6.77 J, 7.12 J, and 5.83 J for TM_TFLite, SM_Scratch_TFLite, and SM_TFLite, respectively. TM_TFLite and SM_Scratch_TFLite have higher energy con-

sumption than that of SM_TFLite, albeit the inference durations of SM_Scratch_TFLite and SM_TFLite were slightly longer than that of TM_TFLite. Consequently, SM_Scratch_TFLite consumed the most energy because it exhibited both high power draw and extended inference duration.

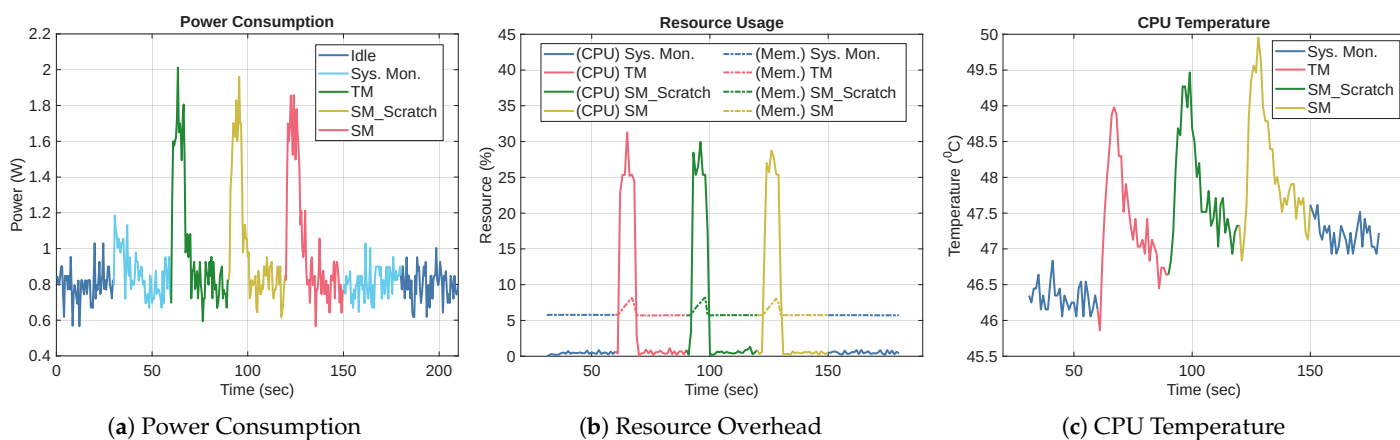


Figure 13. Performance of TM_TFLite, SM_Scratch_TFLite, and SM_TFLite on RPi.

Figure 13b shows CPU and memory utilization, where CPU usage peaked at 31.3%, 29.9%, and 28.7% for TM_TFLite, SM_Scratch_TFLite, and SM_TFLite, respectively, while their respective memory usage peak at 8.16%, 8.20%, and 8.06%. The memory usage trend has a triangular waveform, revealing the model inference pattern. The waveform's incline, peak, and decline correspond to data loading, model inference, and memory deallocation, respectively. The memory waveform is in phase with the power waveform, validating the start and end of the inference window.

Figure 13c depicts the CPU temperature. TM_TFLite reached a maximum temperature of 48.98 °C, SM_Scratch_TFLite reached 49.46 °C, and SM_TFLite peaked at 49.95 °C. Although SM_TFLite has the highest temperature, the difference among the three models is less than 1 °C, indicating negligible thermal variation. Therefore, all three models remained within safe operational limits of a passive cooling system.

6.3.3. Latency Test for TFLite

Table 9 compares the latency of TM_TFLite, SM_Scratch_TFLite, and SM_TFLite across different stages of the inference execution. The results show that TM_TFLite required 10.66 ms to load, while SM_Scratch_TFLite and SM_TFLite loaded much faster at 1.99 ms and 2.04 ms, respectively. This result is intuitive since TM_TFLite is larger than the others. The second and third stages contributed minimally and were nearly identical across all three models. The most notable difference appears in the single-sample inference stage. The results show that SM_Scratch_TFLite and SM_TFLite have the same latency, while TM_TFLite takes twice as long to make a prediction. This latency is due to TM_TFLite being larger than the student models. Moreover, the total latency of TM_TFLite is even worse, 76% slower than the student models. These results demonstrate that KD reduced model size and improved responsiveness on the RPi.

Table 9. Latency breakdown (in milliseconds) for TFLite versions of TM, SM_Scratch, and SM on the RPi.

Stage	TM_TFLite	SM_Scratch_TFLite	SM_TFLite
Load TFLite model	10.66	1.99	2.04
Get input/output tensors	0.15	0.13	0.13
Input data preparation	0.07	0.06	0.06
Single-sample inference	2.84	1.14	1.12
Total latency	13.72	3.32	3.35

6.4. TM Versus Pruning (TM_Prune)

This section evaluates the performance of pruning on the TM using the WDBC dataset. The TM was pruned after training to form TM_Prune. In this experiment, TM_Prune was fine-tuned for an additional 50 epochs to mitigate performance loss.

6.4.1. Accuracy and Size Performance for Pruning

Table 10 shows the results of the experiment, comparing the original TM and pruned TM_Prune. Rows 1 and 2 of Table 10 show the accuracy of TM and TM_Prune before and after conversion to TFLite. The results indicate that pruning degrades accuracy, resulting in a 2.49% drop in performance. Rows 3 and 4 present the file sizes of the two models in Keras and TFLite formats. Since the Keras format stores models without optimization, pruning reduced the file size by 66.35%. However, converting the TM directly to TFLite reduced its size by 66.81%, making it more effective than pruning. Moreover, converting TM_Prune to TFLite resulted in a file size only 1.36% smaller than its Keras counterpart, and the difference between the TFLite formats of TM and TM_Prune was just 0.0041%. From a file size perspective, it is therefore more advantageous to convert the TM to TFLite than to prune it.

Table 10. Performance comparison of TM and TM_Prune in Keras and TFLite formats.

SNo.	Factors	TM	TM_Prune
1.	Accuracy (%)	93.02	90.70
2.	Accuracy (TFLite) (%)	93.02	90.70
3.	Keras Size (kB)	8811.23	2964.86
4.	TFLite Size (kB)	2924.43	2924.31
5.	Total Weights	748,354	748,354
6.	Zero Weights	0	372,864
7.	Sparsity (%)	0.00	49.82

Rows 5–7 present the effect of pruning on the model weights. It reduced the number of active weights by 49.82%, assigning zero to 372,864 parameters. This result has two theoretical benefits: (1) Pruning compressed the TM in its native Keras format from 8811.23 kB to 2964.86 kB, making it easier to store and distribute before deployment. (2) It also introduced sparsity to reduce the inference latency. However, exploiting model sparsity requires specialized hardware, such as GPUs, TPUs, or custom accelerators. Its benefits include reducing computational load and memory access costs.

Comparing the results in Tables 8 and 10 shows that TFLite conversion alone achieves comparable, or even slightly greater, storage reduction than pruning. In addition, pruning requires specialized hardware capable of sparse operations, which can degrade performance and reduce expressiveness [71].

6.4.2. Deployability Test for Pruning

For the deployability test of the pruned models (see Section 5), the experimental procedure is similar to the TFLite experiment, with a few modifications as follows:

1. Record the system for 30 s to establish the baseline power and resource profile of the RPi.
2. Start the system monitor (Sys.Mon.), a Python script that logs CPU temperature, CPU utilization, and memory usage.
3. Execute the TM inference code and wait for the remainder of the 30 s interval.
4. Execute the pruned teacher model (TM_Prune) inference code and wait for the remainder of the 30 s interval.
5. Continue recording for 30 s with only the system monitor running in the background before closing it.
6. Finally, record for an additional 30 s to capture any transient behavior after stopping the system monitor.

Figure 14 presents the power consumption, CPU, and memory performance of TM and TM_Prune averaged over five runs to reduce random error. Figure 14a shows the power consumption profiles of both models. TM reached a maximum of 1.96 W, while TM_Prune peaked at 1.81 W. Although TM_Prune consumed slightly less power, the difference is too small to be considered an improvement. The total energy consumption of TM and TM_Prune was 6.53 J and 5.27 J, respectively. This result shows that TM_Prune consumes less energy than TM. In addition, the TM energy value is consistent with the result reported in Section 6.3.2, validating the reliability of both TM experiments.

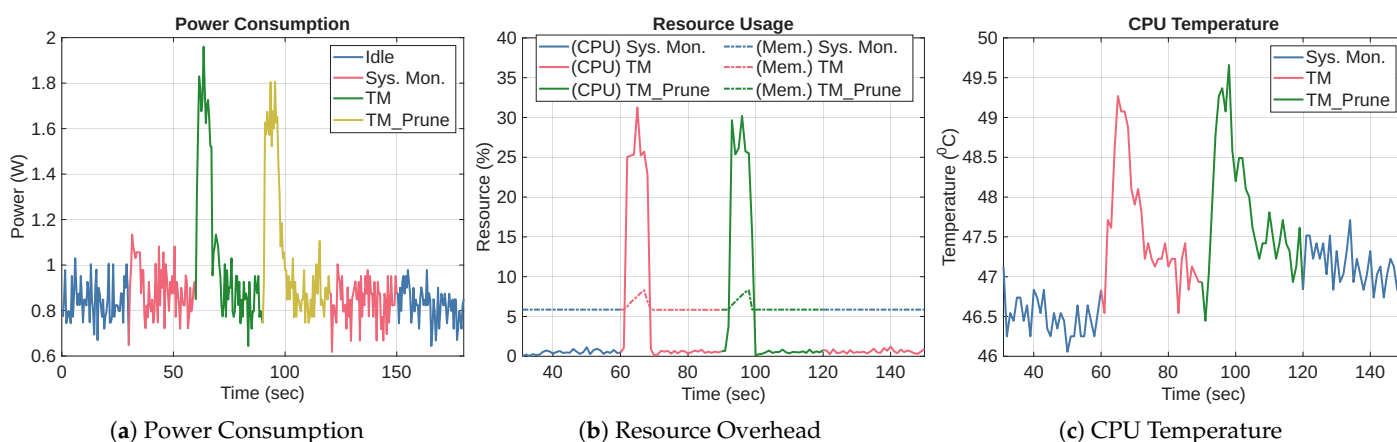


Figure 14. System Performance Metrics During Execution of TM and TM_Prune.

Figure 14b shows the CPU and memory utilization trends. The results show the characteristic pulse waveforms similar to the TFLite experiment in Section 6.3.2. It also shows TM and TM_Prune peaking at 31.26% and 30.2%, respectively. The graph also shows that TM and TM_Prune have CPU usage similar to TM_TFLite, which is higher than SM_Scratch_TFLite and SM_TFLite. In addition, the memory usage waveform follows a similar pattern to the TFLite experiment, peaking at 8.3% for both TM and TM_Prune. Thus, the CPU and memory overhead of TM and TM_Prune are effectively the same as those of TM_TFLite.

Figure 14c illustrates the CPU temperature behavior during the experiment. TM and TM_Prune produced similar thermal responses, with peak temperatures of 49.27 °C and 49.66 °C, respectively, under passive cooling at a room temperature of 23 °C. The result is similar to the heat dissipation of the TFLite experiment, with a change in temperature of TM, TM_Prune, TM_TFLite, SM_Scratch_TFLite, and SM_TFLite being 2.44 °C, 2.74 °C, 2.82 °C,

2.82 °C, and 2.63°C, respectively. Therefore, the temperature changes for all experiments are similar. In conclusion, these results show that pruning led to a slight reduction in power and total energy consumption without any notable change in CPU load, memory usage, or thermal behavior.

6.4.3. Latency Test for Pruning

Table 11 compares the latency of TM and TM_Prune across the different stages of inference on the RPi. We did not report dataset loading latency because it is independent of the models' performance. The TM and TM_Prune load the dataset in 2.23 ms and 2.21 ms, respectively. The table shows that the load time of the TFLite model is nearly identical for both models, with TM at 10.54 ms and TM_Prune slightly higher at 10.72 ms. Similarly, the other stages are almost indistinguishable. The total latency for TM_Prune is 13.68 ms, compared to 13.49 ms for TM, representing no improvement from pruning.

Table 11. Latency breakdown (in milliseconds) for different stages of inference on TM and TM_Prune.

Stage	TM	TM_Prune
Load TFLite model	10.54	10.72
Get input/output tensors	0.15	0.14
Input data preparation	0.06	0.06
Single-sample inference	2.74	2.76
Total latency	13.49	13.68

6.5. TM Versus Post-Training Quantization (TM_PTQ, and TM_PTQ_INT)

In this section, we evaluate the effectiveness of PTQ on the TM using the WDBC dataset. After training the TM, we applied the TFLiteConverter with *converter.optimizations = [tf.lite.Optimize.DEFAULT]* to create TM_PTQ, a weight-only quantized version of the TM. We also applied full-integer quantization to TM to produce TM_PTQ_INT.

6.5.1. Accuracy and Size Performance for PTQ

Table 12 shows the accuracy and file sizes of TM, TM_PTQ, and TM_PTQ_INT. The results show that TM_PTQ preserves the original TM's accuracy at 96.51%, demonstrating that weight-only quantization has no impact on predictive performance on the WDBC dataset. In contrast, the integer-only TM_PTQ_INT shows a 2.32 percentage points reduction in accuracy. This reduction is expected because converting all parameters and activations to integers introduces quantization error and reduces numerical precision. It illustrates the trade-off inherent in full-integer quantization. On the one hand, it enables the model to run entirely in the integer domain, which is necessary for deployment on some resource-constrained hardware. On the other hand, the conversion degrades accuracy.

Table 12. Performance comparison of TM, TM_PTQ, and TM_PTQ_INT.

Factor	TM	TM_PTQ	TM_PTQ_INT
Accuracy (%)	96.51	96.51	94.19
TFLite Size (kB)	2925.24	746.59	750.69

The second row of Table 12 shows the TFLite file sizes for TM, TM_PTQ, and TM_PTQ_INT. It shows that TM_PTQ and TM_PTQ_INT are 74.48% and 74.34% smaller than TM, respectively. Interestingly, TM_PTQ_INT is 4.10 kB larger than TM_PTQ despite being fully quantized. To investigate this counter-intuitive result, we implemented a Python script (see *tflite_visualization.ipynb* in [63]) to analyze the contents of the two files. The breakdown is provided in Table 13. It shows that TM_PTQ_INT is larger because it contains

two additional buffers and one extra operator code compared to TM_PTQ. Both models rely on the same core operators (CONV_2D, FULLY_CONNECTED, MUL, ADD, RESHAPE, LOGISTIC, and SOFTMAX) to implement the CNN layers and activations.

However, TM_PTQ_INT includes an additional QUANTIZE operator (opcode 114), which ensures that inputs and activations remain in integer form, thereby enabling true integer-only inference. Hence, the overhead in TM_PTQ_INT arises from quantization metadata and the extra QUANTIZE operator, rather than from storing weights. Although TM_PTQ_INT performs worse than TM_PTQ, it is applicable in real-time IoT systems where IoT devices support only integer operations. However, for platforms that support mixed precision, TM_PTQ may be a more suitable choice, as it preserves accuracy while substantially reducing model size.

Table 13. TFLite file content analysis of TM_PTQ and TM_PTQ_INT.

Feature	TM_PTQ	TM_PTQ_INT
Total file size (bytes)	764,504	768,704
Number of subgraphs	1	1
Number of buffers	32	34
Total buffer size (bytes)	753,032	749,192
Largest buffer (bytes)	688,128	688,128
Operator codes	7	8
Buffers (% of file)	98.50%	97.46%
Other metadata & structure (%)	1.50%	2.54%

6.5.2. Deployability Test for PTQ

The deployability test for the PTQ experiment is similar to the TFLite deployability experiment in Section 6.3.2 with a few modifications as follows:

1. Record the system for 30 s to obtain the baseline profile of the RPi.
2. Start the system monitor (Sys.Mon.), a Python script that logs CPU temperature along with CPU and memory usage.
3. Execute the TM inference code and wait for the remainder of the 30 s.
4. Execute the TM_PTQ inference code and wait for the remainder of the 30 s.
5. Execute the TM_PTQ_INT inference code and wait for the remainder of the 30 s.
6. Continue recording for 30 s with only the system monitor running in the background before closing it.
7. Finally, record for another 30 s to capture any transient responses after closing the system monitor.

Figure 15 presents the CPU and memory performance of TM, TM_PTQ, and TM_PTQ_INT on the RPi. The figures were plotted from the average of 5 samples to reduce the random error. Figure 15a is the power consumption over time graph of the three models. It shows TM, TM_PTQ, and TM_PTQ_INT peak power consumption at 2.81 W, 2.03 W, and 2.06 W, respectively. The PTQ-based models consumed less power than the baseline TM. However, the difference between TM_PTQ and TM_PTQ_INT is negligible. The total energy consumption was 10.29 J, 6.37 J, and 7.10 J for TM, TM_PTQ, and TM_PTQ_INT, respectively. They indicate that the PTQ techniques substantially reduce energy usage compared to TM, and the TM_PTQ_INT is slightly less efficient than TM_PTQ.

Figure 15b shows the CPU and memory overhead of the three models. The results indicate that CPU usage peaked at 31.12%, 29.98%, and 30.8% for TM, TM_PTQ, and TM_PTQ_INT, respectively. Their respective peak memory usage was nearly the same at 8.56%, 8.52%, and 8.44%. The plots show the familiar pulse and triangular waveforms. Also, the CPU temperature overhead is shown in Figure 15c. It shows that TM, TM_PTQ,

and TM_PTQ_INT have maximum temperatures of 50.34 °C, 50.83 °C, and 51.32 °C, respectively. The temperature values are too close to conclude that one model is better than the others. All models remained within safe operating limits under passive cooling. Thus, PTQ considerably improves energy efficiency without compromising performance stability or increasing computational overhead.

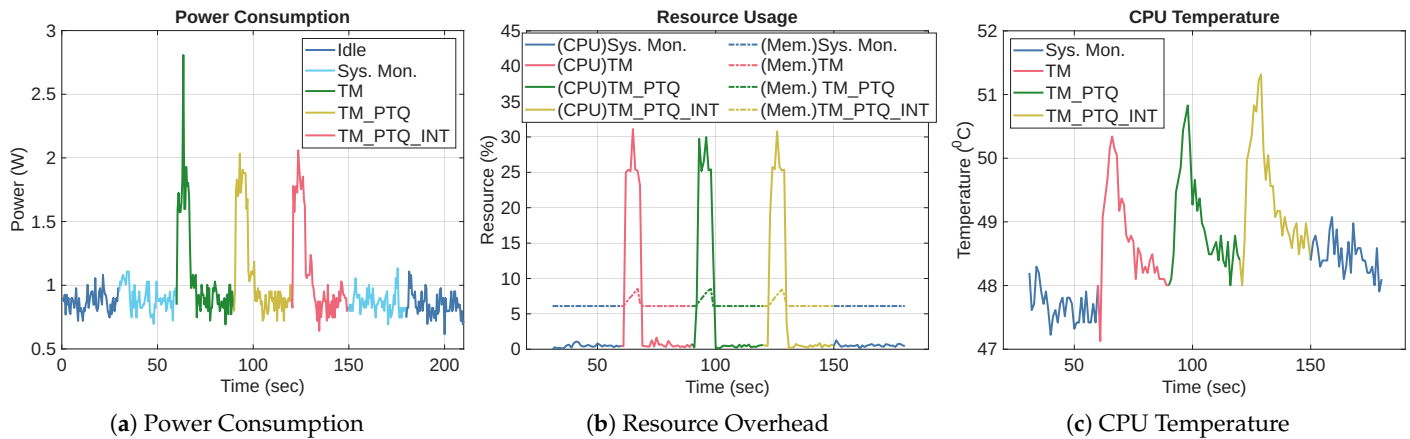


Figure 15. System Performance Metrics During Execution of TM, TM_PTQ, and TM_PTQ_INT.

6.5.3. Latency Test for PTQ

Table 14 presents the latency breakdown for TM, TM_PTQ, and TM_PTQ_INT across different stages of inference on the RPi. As in the previous experiment, model loading dominates the latency. The full-precision TM requires 10.58 ms to load, while TM_PTQ and TM_PTQ_INT load much faster, at 2.20 ms and 1.04 ms, respectively. The subsequent stages (i.e., obtaining input/output tensors, preparing input data, and performing single-sample inference) contribute only marginally to the total, and their values remain similar across all three models. The total latency for TM_PTQ is 5.07 ms, and for TM_PTQ_INT is 4.10 ms, compared to 13.52 ms for TM. This represents a 62.5% reduction for TM_PTQ and a 69.7% reduction for TM_PTQ_INT relative to TM.

Table 14. Latency (in milliseconds) for different stages of inference on TM, TM_PTQ, and TM_PTQ_INT.

Stage	TM	TM_PTQ	TM_PTQ_INT
Load TFLite model	10.58	2.20	1.04
Get input/output tensors	0.14	0.13	0.11
Input data preparation	0.12	0.06	0.23
Single-sample inference	2.68	2.68	2.72
Total latency	13.52	5.07	4.10

6.6. TM Versus Quantization-Aware Training (TM_QAT)

In this section, we evaluate the effectiveness of TM optimized using QAT (TM_QAT) and trained on the WDBC dataset. We compared the performance of TM_QAT against the baseline TM in terms of accuracy, file size, and deployment suitability.

6.6.1. Accuracy and Size Performance for QAT

Table 15 shows the accuracy and change in file size for TM and TM_QAT. The table did not report sparsity because QAT does not affect the model's weights or neurons. The first row shows that TM achieves 95.35% accuracy, and TM_QAT reaches 93.02%, resulting in a 2.33% reduction. This decline in accuracy is expected because of quantization error [55].

Quantization error is the inherent loss of precision when converting high-precision data formats to lower-precision ones. Nevertheless, the model is useful because TM_QAT's accuracy is high enough for some medical applications.

Table 15. Performance comparison of TM and TM_QAT.

Factor	TM	TM_QAT
Accuracy (%)	95.35	93.02
Keras size (kB)	8811.46	9057.71
TFLite size (kB)	2925.24	737.69

The second row of Table 15 compares the Keras file sizes of TM and TM_QAT. The results show that TM is 8811.46 kB, while TM_QAT is slightly larger at 9057.71 kB with an increase of 2.80%. This increase is due to QAT introducing additional quantization metadata and simulated quantization nodes during training. However, this increase does not reflect additional model complexity in terms of layers or parameters. Although the Keras version of TM_QAT is larger, its TFLite version is significantly lighter. As shown in the third row, the TFLite size of TM is 2925.24 kB, whereas TM_QAT compresses to 737.69 kB, resulting in a reduction of approximately 74.8%.

6.6.2. Deployability Test for QAT

This section presents the deployability test on the QAT models using the setup in Section 5. The experimental procedure for this experiment is as outlined below:

1. Record the system for 30 s to establish the baseline power and resource profile of the RPi.
2. Start the system monitor (Sys.Mon.), a Python script that logs CPU temperature, CPU utilization, and memory usage.
3. Execute the TM inference code and wait for the remainder of the 30 s interval.
4. Execute the pruned teacher model (TM_QAT) inference code and wait for the remainder of the 30 s interval.
5. Continue recording for 30 s with only the system monitor running in the background before closing it.
6. Finally, record for an additional 30 s to capture any transient behavior after stopping the system monitor.

Figure 16 presents the power consumption, CPU, and memory usage of TM_QAT in comparison to TM on the RPi setup. Figure 16a depicts the power consumption profiles of both models. TM reached a peak power of 1.98 W, whereas TM_QAT peaked at 1.88 W. Additionally, the total energy consumption was 5.88 J for TM and 5.62 J for TM_QAT. Although TM_QAT exhibited slightly lower total energy consumption than TM, this difference is insufficient to represent a significant improvement.

Figure 16b presents CPU and memory utilization. The results show comparable resource usage between the two models, with CPU utilization peaking at 31.26% for TM and 29.32% for TM_QAT, while memory usage remained steady at 8.54% and 8.50%, respectively. The slight variation in the results indicates that quantization-aware training does not substantially alter computational demand or resource utilization on this platform. Similarly, Figure 16c depicts the CPU temperature profiles. TM reached a maximum of 50.05 °C, while TM_QAT peaked slightly higher at 50.63 °C. Thus, there is no difference in temperature between the two.

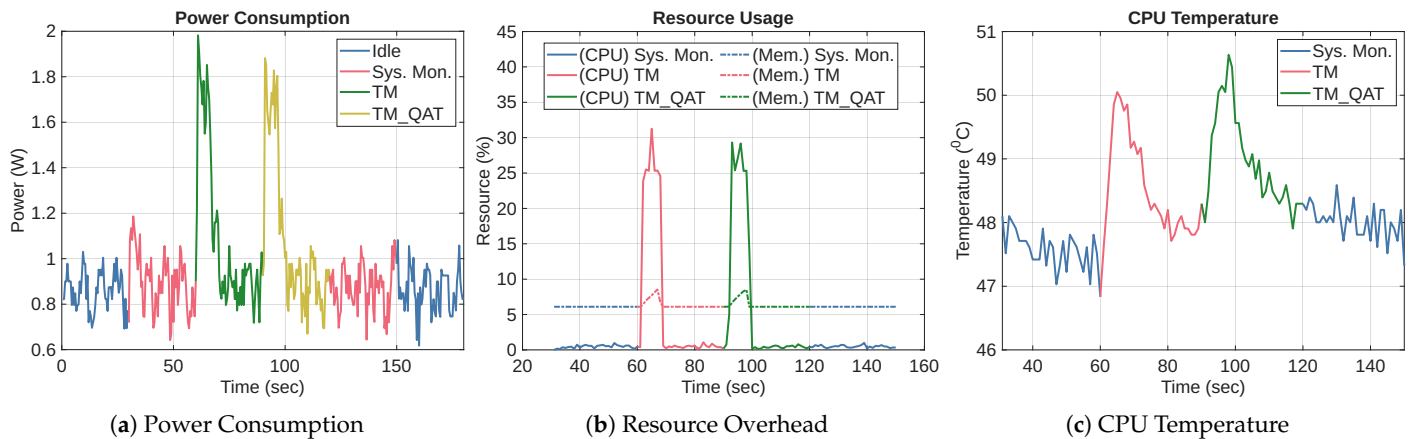


Figure 16. System Performance Metrics During Execution of TM and TM_QAT.

6.6.3. Latency Test for QAT

Table 16 compares the latency of TM and TM_QAT. The results show that model loading dominates the total latency, with TM requiring 10.72 ms compared to only 1.14 ms for TM_QAT. This confirms that quantization-aware training results in a more compact, faster-to-initialize model. The total latency of TM_QAT is 4.05 ms, which is approximately 70% lower than TM's 13.63 ms. These results demonstrate that, as with PTQ, the QAT technique provides significant reductions in initialization and execution time while maintaining nearly identical inference latency per sample.

Table 16. Latency breakdown (in milliseconds) for different stages of inference on TM and TM_QAT.

Stage	TM	TM_QAT
Load TFLite model	10.72	1.14
Get input/output tensors	0.14	0.11
Input data preparation	0.07	0.21
Single-sample inference	2.70	2.59
Total latency	13.63	4.05

6.7. Summary of Findings

This section summarizes what we learned from the results. The results show that SM_TFLite achieved 97.67%, incurring a 2.33 percentage point drop relative to its TM, which is similar to the losses seen in TM_Prune (90.70%, −2.32 points), TM_PTQ_INT (94.19%, −2.32 points), and TM_QAT (93.02%, −2.33 points). Only TM_PTQ preserved complete baseline accuracy at 96.51%. In terms of storage, SM_TFLite was by far the most compact, requiring just 5.21 kB, compared to 2924.31 kB for TM_Prune, 746.59 kB for TM_PTQ, 750.69 kB for TM_PTQ_INT, and 737.69 kB for TM_QAT. Thus, SM_TFLite is the smallest because of the combination of its fewer parameters, 582 parameters compared to the TM's 863,042 (99.93% fewer parameters), in addition to the TFLite optimizations.

Moreover, latency results further confirmed the advantage of SM_TFLite, which achieved the lowest total inference time at 3.3 ms, outperforming TM_PTQ (5.07 ms), TM_PTQ_INT (4.10 ms), TM_QAT (4.05 ms), and TM_Prune (13.68 ms), while the baseline TM required 13.52 ms. Resource usage experiments showed that all models consumed similar CPU and memory overhead. The temperature variation between the models is too slight to draw a definite conclusion. The highest temperature change was observed in TM_PTQ_INT at 3.31 °C (51.32–48.00 °C) while the lowest temperature was from TM during the pruning experiment at 2.44 °C (49.66–46.93 °C), making the variation only

0.87 °C. The slight variations between the models mean they have similar temperature performance. Taken together, these findings demonstrate that KD combined with TFLite (SM_TFLite) conversion delivers the most effective balance among the tested techniques, making it the most suitable strategy for lightweight BCD deployment on IoT platforms.

7. Limitation of the Study

This study used three publicly available datasets that are relatively small and primarily numerical rather than image-based, which may limit generalizability to complete clinical screening workflows. Also, the PBCNT dataset is high-dimensional and highly imbalanced, increasing the risk of overfitting despite cross-validation and oversampling. Furthermore, since all experiments were conducted on text- and tabular-domain data, an image-processing algorithm is required to ensure transferability to real-world applications. However, this will increase the system latency.

Furthermore, the proposed SM_TFLite model incurred only 2.33% accuracy loss relative to its teacher model, despite demonstrating the best overall balance between accuracy, latency, and compactness. Its extreme compactness (5.21 kB, 99.93% fewer parameters) may reduce representational flexibility, potentially limiting performance on more complex or heterogeneous data. Its efficiency depends on TFLite optimizations validated only on the RPi. Therefore, broader hardware scalability and robustness under real-world operational workloads are required. Finally, the cross-dataset experiment showed a substantial performance drop, highlighting that the models still struggle with significant domain shift between disparate datasets.

8. Conclusions and Future Work

The main goal of this study was to develop a lightweight CNN capable of effectively identifying BC by emulating the performance of a DNN, enabling deployment on low-cost embedded devices for LMIC healthcare systems. Statistical analysis shows that SM performs comparably to TM and significantly better than SM_Scratch at the 95% confidence interval. The proposed system uses a text-based dataset to detect BC with an accuracy of 97–100% across three benchmark datasets while reducing the number of parameters by more than 87%. Experiments on the RPi 4 Model B confirm that the proposed model is deployable on embedded systems, achieving inference times as low as 1.12 ms, faster than all other evaluated techniques. It also demonstrates low memory overhead (less than 8.5% of 8 GB), similar to pruning and quantization. These results make the model suitable for most telemedicine applications.

The resource experiments also show CPU utilization below 32% and a temperature increase below 3.31 °C across all techniques, indicating that KD is indistinguishable from pruning and quantization in terms of computational overhead. Among all lightweight conversion methods, SM_TFLite delivered the most favorable balance, offering accuracy on par with pruning, QAT, and full-integer PTQ, and only slightly lower than dynamic-range PTQ. It achieved the smallest file size and the shortest inference time. The main advantages of the proposed model are its exceptional accuracy, fast processing speed, minimal trainable parameters, and the ability to be deployed on low-power, resource-constrained devices such as laptops, single-board computers, tablets, and mobile phones. Therefore, this research has the potential to enhance telemedicine in oncology, especially in remote and underserved regions where access to high-performance computing is limited.

In the future, we plan to extend the proposed KD framework to architectures designed for image-based datasets. This adaptation will enable broader validation across different data modalities and support the application of the method to more complex medical

imaging tasks. Additionally, we aim to implement and evaluate this model on low-power devices for real-world healthcare applications.

Author Contributions: Conceptualization, R.P.; Data curation, F.M.; Formal analysis, F.M.; Funding acquisition, F.M.; Investigation, F.M.; Methodology, F.A.; Project administration, R.P.; Resources, F.M.; Supervision, R.P.; Validation, R.P. and F.A.; Visualization, F.M. and F.A.; Writing—original draft, F.M.; Writing—review & editing, F.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research and APC were funded by Google, through the Google Ph.D. Fellowship program.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The source code developed for this study is available at <https://github.com/falmatamkn/KD-4-BCD> (accessed on 5 December 2025). The datasets used (PBCNT, BCDD, and WDBC) are publicly accessible through their respective repositories, and the dataset locations are listed in the GitHub repository and referenced in the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
AUC	Area Under the Curve
APC	Article Processing Charge
BC	Breast Cancer
BCD	Breast Cancer Detection
BCDD	Breast Cancer Dataset
CI	Confidence Interval
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DL	Deep Learning
DTN	Delay-Tolerant Network
FL	Federated Learning
FN	False Negative
FP	False Positive
FPR	False Positive Rate
FNA	Fine Needle Aspirate
GPU	Graphics Processing Unit
IoT	Internet of Things
KD	Knowledge Distillation
KL	Kullback–Leibler
LMICs	Low- and Middle-Income Countries
ML	Machine Learning
PBCNT	Primary Breast Cancer vs. Normal Breast Tissue Dataset
PCA	Principal Component Analysis
PTQ	Post-Training Quantization
QAT	Quantization-Aware Training
RAM	Random Access Memory
ReLU	Rectified Linear Unit

RPi	Raspberry Pi
SM	Student Model
SVM	Support Vector Machine
TFLite	TensorFlow Lite
TM	Teacher Model
TN	True Negative
TP	True Positive
TPR	True-Positive Rate
WDBC	Wisconsin Diagnostic Breast Cancer Dataset

References

1. WHO. Breast Cancer. 2025. Available online: <https://www.who.int/news-room/fact-sheets/detail/breast-cancer> (accessed on 5 December 2025).
2. American Cancer Society. About Breast Cancer. 2022. Available online: <https://www.cancer.org/content/dam/CRC/PDF/Public/8577.00.pdf> (accessed on 5 December 2025).
3. Delalogue, S.; Khan, S.A.; Wesseling, J.; Whelan, T. Ductal carcinoma in situ of the breast: Finding the balance between overtreatment and undertreatment. *Lancet* **2024**, *403*, 2734–2746. [CrossRef] [PubMed]
4. Kim, J.; Harper, A.; McCormack, V.; Sung, H.; Houssami, N.; Morgan, E.; Mutebi, M.; Garvey, G.; Soerjomataram, I.; Fidler-Benaoudia, M.M. Global patterns and trends in breast cancer incidence and mortality across 185 countries. *Nat. Med.* **2025**, *31*, 1154–1162. [CrossRef] [PubMed]
5. Shen, Y.; Shamout, F.E.; Oliver, J.R.; Witowski, J.; Kannan, K.; Park, J.; Wu, N.; Huddleston, C.; Wolfson, S.; Millet, A.; et al. Artificial intelligence system reduces false-positive findings in the interpretation of breast ultrasound exams. *Nat. Commun.* **2021**, *12*, 5645. [CrossRef] [PubMed]
6. Eisemann, N.; Bunk, S.; Mukama, T.; Baltus, H.; Elsner, S.A.; Gomille, T.; Hecht, G.; Heywang-Köbrunner, S.; Rathmann, R.; Siegmann-Luz, K.; et al. Nationwide real-world implementation of AI for cancer detection in population-based mammography screening. *Nat. Med.* **2025**, *31*, 917–924. [CrossRef]
7. WHO. State of Digital Health Around the World Today. 2023. Available online: https://monitor.digitalhealthmonitor.org/country_list (accessed on 17 October 2025).
8. Manson, E.N.; Achel, D.G. Fighting breast cancer in low-and-middle-income countries – What must we do to get every woman screened on regular basis? *Sci. Afr.* **2023**, *21*, e01848. [CrossRef]
9. Chia, J.L.L.; He, G.S.; Ngiam, K.Y.; Hartman, M.; Ng, Q.X.; Goh, S.S.N. Harnessing Artificial Intelligence to Enhance Global Breast Cancer Care: A Scoping Review of Applications, Outcomes, and Challenges. *Cancers* **2025**, *17*, 197. [CrossRef]
10. Wang, C.H.; Huang, K.Y.; Yao, Y.; Chen, J.C.; Shuai, H.H.; Cheng, W.H. Lightweight Deep Learning: An Overview. *IEEE Consum. Electron. Mag.* **2022**, *13*, 51–64. [CrossRef]
11. Modu, F.; Prasad, R.; Aliyu, F. Lightweight CNN for Resource-Constrained BCD System Using Knowledge Distillation. *IEEE Access* **2025**, *13*, 57504–57529. [CrossRef]
12. Li, Y.; Mercan, E.; Knezevich, S.; Elmore, J.G.; Shapiro, L.G. Efficient and Accurate Mitosis Detection-A Lightweight RCNN Approach. In Proceedings of the ICPRAM, Funchal, Portugal, 16–18 January 2018; SciTePress: Setúbal, Portugal, 2018; Volume 1, pp. 69–77. [CrossRef]
13. Kim, K.; Jang, S.J.; Park, J.; Lee, E.; Lee, S.S. Lightweight and Energy-Efficient Deep Learning Accelerator for Real-Time Object Detection on Edge Devices. *Sensors* **2023**, *23*, 1185. [CrossRef]
14. Ahmad, B.; Jun, S.; Li, J.; Lidan, B. Comparative Study of Deep Learning Models For Automatic Detection of Metastases in H&E Stained Images. In Proceedings of the 2021 4th International Conference on Computing & Information Sciences (ICCIS), Karachi, Pakistan, 29–30 November 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–6. [CrossRef]
15. Pramanik, R.; Pramanik, P.; Sarkar, R. Breast cancer detection in thermograms using a hybrid of GA and GWO based deep feature selection method. *Expert Syst. Appl.* **2023**, *219*, 119643. [CrossRef]
16. Laxmisagar, H.S.; Hanumantharaju, M.C. Design of an Efficient Deep Neural Network for Multi-level Classification of Breast Cancer Histology Images. In *Intelligent Computing and Applications*; Dash, S.S., Das, S., Panigrahi, B.K., Eds.; Springer: Singapore, 2021; pp. 447–459.
17. Laxmisagar, H.; Hanumantharaju, M. Detection of Breast Cancer with Lightweight Deep Neural Networks for Histology Image Classification. *Crit. Rev. Biomed. Eng.* **2022**, *50*, 1–19. [CrossRef] [PubMed]
18. Abdelli, A.; Saouli, R.; Djemal, K.; Youkana, I. Combined Datasets For Breast Cancer Grading Based On Multi-CNN Architectures. In Proceedings of the 2020 Tenth International Conference on Image Processing Theory, Tools and Applications (IPTA), Paris, France, 9–12 November 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–7. [CrossRef]

19. Liu, S.; Ren, J.; Chen, Z.; Hu, K.; Xiao, F.; Li, X.; Gao, X. EffiDiag: An Efficient Framework for Breast Cancer Diagnosis in Multi-Gigapixel Whole Slide Images. In Proceedings of the 2020 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), Seoul, Republic of Korea, 16–19 December 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 663–669. [\[CrossRef\]](#)
20. Oliveira, H.S.; Teixeira, J.F.; Oliveira, H.P. Lightweight Deep Learning Pipeline for Detection, Segmentation and Classification of Breast Cancer Anomalies. In Proceedings of the Image Analysis and Processing—ICIAP 2019, Trento, Italy, 9–13 September 2019; Ricci, E., Rota Bulò, S., Snoek, C., Lanz, O., Messelodi, S., Sebe, N., Eds.; Springer: Cham, Switzerland, 2019; pp. 707–715.
21. Elbashir, M.K.; Ezz, M.; Mohammed, M.; Saloum, S.S. Lightweight Convolutional Neural Network for Breast Cancer Classification Using RNA-Seq Gene Expression Data. *IEEE Access* **2019**, *7*, 185338–185348. [\[CrossRef\]](#)
22. Kim, S.; Yoon, K. Deep and Lightweight Neural Network for Histopathological Image Classification. *J. Mob. Multimed.* **2022**, *18*, 1913–1930. [\[CrossRef\]](#)
23. Garg, S.; Singh, P. Transfer Learning Based Lightweight Ensemble Model for Imbalanced Breast Cancer Classification. *IEEE/ACM Trans. Comput. Biol. Bioinform.* **2023**, *20*, 1529–1539. [\[CrossRef\]](#)
24. Yan, C.; Zeng, X.; Xi, R.; Ahmed, A.; Hou, M.; Tunio, M.H. PLA—A Privacy-Embedded Lightweight and Efficient Automated Breast Cancer Accurate Diagnosis Framework for the Internet of Medical Things. *Electronics* **2023**, *12*, 4923. [\[CrossRef\]](#)
25. Nneji, G.U.; Monday, H.N.; Mgbejime, G.T.; Pathapati, V.S.R.; Nahar, S.; Ukwuoma, C.C. Lightweight Separable Convolution Network for Breast Cancer Histopathological Identification. *Diagnostics* **2023**, *13*, 299. [\[CrossRef\]](#)
26. Kausar, T.; Lu, Y.; Kausar, A. Breast Cancer Diagnosis Using Lightweight Deep Convolution Neural Network Model. *IEEE Access* **2023**, *11*, 124869–124886. [\[CrossRef\]](#)
27. Yan, T.; Chen, G.; Zhang, H.; Wang, G.; Yan, Z.; Li, Y.; Xu, S.; Zhou, Q.; Shi, R.; Tian, Z.; et al. Convolutional neural network with parallel convolution scale attention module and ResCBAM for breast histology image classification. *Heliyon* **2024**, *10*, e30889. [\[CrossRef\]](#)
28. Kotei, E.; Thirunavukarasu, R. Visual attention condenser model for multiple disease detection from heterogeneous medical image modalities. *Multimed. Tools Appl.* **2024**, *83*, 30563–30585. [\[CrossRef\]](#)
29. Kalita, M.; Mahanta, L.B.; Das, A.K.; Nath, M. A new deep learning model with interface for fine needle aspiration cytology image-based breast cancer detection. *Indones. J. Electr. Eng. Comput. Sci.* **2024**, *34*, 1739–1752. [\[CrossRef\]](#)
30. Zhang, X.; Lin, X.; Zhang, Z.; Dong, L.; Sun, X.; Sun, D.; Yuan, K. Artificial Intelligence Medical Ultrasound Equipment: Application of Breast Lesions Detection. *Ultrason. Imaging* **2020**, *42*, 191–202. [\[CrossRef\]](#) [\[PubMed\]](#)
31. Wang, Y.; Wang, Y.; Cai, J.; Lee, T.K.; Miao, C.; Wang, Z.J. Ssd-kd: A self-supervised diverse knowledge distillation method for lightweight skin lesion classification using dermoscopic images. *Med Image Anal.* **2023**, *84*, 102693. [\[CrossRef\]](#) [\[PubMed\]](#)
32. Aboutahoun, D.; Zewail, R.; Kimura, K.; Soliman, M.I. Lightweight Histological Tumor Classification Using a Joint Sparsity-Quantization Aware Training Framework. *IEEE Access* **2023**, *11*, 119342–119351. [\[CrossRef\]](#)
33. Si, J.; Jia, W.; Jiang, H. Breast cancer histopathology images classification through multi-view augmented contrastive learning and pre-learning knowledge distillation. *IEEE Access* **2024**, *12*, 25359–25371. [\[CrossRef\]](#)
34. Wang, L.; Lu, H. Classification of histopathologic images of breast cancer by multi-teacher small-sample knowledge distillation. In Proceedings of the 2021 2nd International Conference on Artificial Intelligence and Computer Engineering (ICAICE), Hangzhou, China, 5–7 November 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 642–647.
35. Ashraf, F.B.; Alam, S.M.; Sakib, S.M. Enhancing breast cancer classification via histopathological image analysis: Leveraging self-supervised contrastive learning and transfer learning. *Heliyon* **2024**, *10*, e24094. [\[CrossRef\]](#)
36. Chaudhury, S.; Sau, K.; Khan, M.A.; Shabaz, M. Deep transfer learning for IDC breast cancer detection using fast AI technique and SqueezeNet architecture. *Math. Biosci. Eng.* **2023**, *20*, 10404–10427. [\[CrossRef\]](#)
37. Sabih, M.; Hannig, F.; Teich, J. DyFiP: Explainable AI-Based Dynamic Filter Pruning of Convolutional Neural Networks. In Proceedings of the 2nd European Workshop on Machine Learning and Systems, Rennes, France, 5–8 April 2022; Association for Computing Machinery: New York, NY, USA, 2022; EuroMLSys '22; pp. 109–115. [\[CrossRef\]](#)
38. Kumar, A.; Sharma, A.; Bharti, V.; Singh, A.K.; Singh, S.K.; Saxena, S. MobiHisNet: A lightweight CNN in mobile edge computing for histopathological image classification. *IEEE Internet Things J.* **2021**, *8*, 17778–17789. [\[CrossRef\]](#)
39. Nissar, I.; Alam, S.; Masood, S.; Kashif, M. MOB-CBAM: A dual-channel attention-based deep learning generalizable model for breast cancer molecular subtypes prediction using mammograms. *Comput. Methods Programs Biomed.* **2024**, *248*, 108121. [\[CrossRef\]](#)
40. Raiaan, M.A.K.; Fahad, N.M.; Mukta, M.S.H.; Shatabda, S. Mammo-Light: A lightweight convolutional neural network for diagnosing breast cancer from mammography images. *Biomed. Signal Process. Control* **2024**, *94*, 106279. [\[CrossRef\]](#)
41. Patel, W.; Koyuncu, H.; Ganatara, A. Revolutionizing Cancer Diagnosis in IoMT with a Novel Lightweight Deep Learning Model for Histopathological Image Classification. In Proceedings of the 2023 International Conference on Modeling, Simulation & Intelligent Computing (MoSiCom), Dubai, United Arab Emirates, 7–9 December 2023; pp. 252–257. [\[CrossRef\]](#)
42. Kang, M.; Kang, S. Knowledge distillation with insufficient training data for regression. *Eng. Appl. Artif. Intell.* **2024**, *132*, 108001. [\[CrossRef\]](#)

43. Bergmann, D. What Is Knowledge Distillation? 2024. Available online: <https://www.ibm.com/topics/knowledge-distillation> (accessed on 20 October 2024).
44. Intel Labs. Knowledge distillation. In *Distiller Documentation*; Intel ®: Santa Clara, CA, USA, 2019; Chapter 13; pp. 13–14. Available online: https://intellabs.github.io/distiller/knowledge_distillation.html (accessed on 20 October 2024).
45. Chariton, A. Knowledge Distillation Tutorial—PyTorch Tutorials 2.5.0+cu124 Documentation. 2024. Available online: https://pytorch.org/tutorials/beginner/knowledge_distillation_tutorial.html (accessed on 20 October 2024).
46. Borup, K. Keras Documentation: Knowledge Distillation. 2020. Available online: https://keras.io/examples/vision/knowledge_distillation/ (accessed on 20 October 2024).
47. Hinton, G.; Vinyals, O.; Dean, J. Distilling the knowledge in a neural network. *arXiv* **2015**, arXiv:1503.02531. [CrossRef]
48. Vadera, S.; Ameen, S. Methods for Pruning Deep Neural Networks. *IEEE Access* **2022**, *10*, 63280–63300. [CrossRef]
49. TensorFlow. 2023. Available online: <http://bit.ly/498YyWc> (accessed on 17 October 2025).
50. Gholami, A.; Kim, S.; Dong, Z.; Yao, Z.; Mahoney, M.W.; Keutzer, K. A Survey of Quantization Methods for Efficient Neural Network Inference. In *Low-Power Computer Vision: Improve the Efficiency of Artificial Intelligence*; Thiruvathukal, G.K., Lu, Y.H., Kim, J., Chen, Y., Chen, B., Eds.; Chapman and Hall/CRC: New York, NY, USA, 2022; Chapter 13; p. 36.
51. Sumanth, P. Model Quantization: Meaning, Benefits & Techniques. 2025. Available online: <https://www.clarifai.com/blog/model-quantization> (accessed on 18 October 2025).
52. Krishnamoorthi, R.; Reed, J.; Ni, M.; Gottbrath, C.; Weidman, S. Introduction to Quantization on PyTorch. 2020. Available online: <https://pytorch.org/blog/introduction-to-quantization-on-pytorch/> (accessed on 18 October 2025).
53. Cheng, W.L. Introducing Post-Training Model Quantization Feature and Mechanics Explained. 2024. Available online: <https://datature.io/blog/introducing-post-training-quantization-feature-and-mechanics-explained> (accessed on 2 November 2025).
54. TensorFlow. Post-Training Quantization: Tensorflow Model Optimization. 2022. Available online: <http://bit.ly/3WHPBf1> (accessed on 18 October 2025).
55. Clark, B. What Is Quantization Aware Training? 2025. Available online: <https://www.ibm.com/think/topics/quantization-aware-training> (accessed on 2 November 2025).
56. TensorFlow. Quantization Aware Training Comprehensive Guide. 2025. Available online: <http://bit.ly/48ClewU> (accessed on 18 October 2025).
57. Matamala, N.; Vargas, M.T.; González-Cámpora, R.; Miñambres, R.; Arias, J.I.; Menéndez, P.; Andrés-León, E.; Gómez-López, G.; Yanowsky, K.; Calvete-Candenas, J.; et al. Tumor microRNA expression profiling identifies circulating microRNAs for early breast cancer detection. *Clin. Chem.* **2015**, *61*, 1098–1106. [CrossRef] [PubMed]
58. Matamala, N.; Vargas, M.; González-Cámpora, R.; Miñambres, R. Primary Breast Cancer vs Normal Breast Tissue. 2015. Available online: <https://www.kaggle.com/datasets/rhostam/primary-breast-cancer-vs-normal-breast-tissue> (accessed on 2 November 2025).
59. Wolberg, W.H.; Mangasarian, O.L. Multisurface method of pattern separation for medical diagnosis applied to breast cytology. *Proc. Natl. Acad. Sci. USA* **1990**, *87*, 9193–9196. [CrossRef]
60. Rouse, B. Wisconsin Breast Cancer Database. 2017. Available online: <https://www.kaggle.com/datasets/roustekbio/breast-cancer-csv> (accessed on 2 November 2025).
61. Bennett, K.P.; Mangasarian, O.L. Robust Linear Programming Discrimination of Two Linearly Inseparable Sets. *Optim. Methods Softw.* **1992**, *1*, 23–34. [CrossRef]
62. Wolberg, W.; Mangasarian, O.; Street, N.; Street, W. Breast Cancer Wisconsin (Diagnostic) Data Set. 2016. Available online: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data> (accessed on 2 November 2025).
63. Modu, F. Breast Cancer Detection (BCD) Using Knowledge Distillation (KD). 2024. Available online: <https://github.com/falmatamkn/KD-4-BCD> (accessed on 2 November 2024).
64. Masa-Ibi, E.; Prasad, R. Breast Cancer Classification Using Discrete Wavelet Transformation and Deep Learning. *Recent Adv. Comput. Sci. Commun.* **2021**, *14*, 2103–2112. [CrossRef]
65. Naji, M.A.; El Filali, S.; Aarika, K.; Benlahmar, E.H.; Abdelouhahid, R.A.; Debauche, O. Machine learning algorithms for breast cancer prediction and diagnosis. *Procedia Comput. Sci.* **2021**, *191*, 487–492. [CrossRef]
66. Hajiabadi, H.; Babaiyan, V.; Zabihzadeh, D.; Hajiabadi, M. Combination of loss functions for robust breast cancer prediction. *Comput. Electr. Eng.* **2020**, *84*, 106624. [CrossRef]
67. Kumar, A.; Saini, R.; Kumar, R. A Comparative Analysis of Machine Learning Algorithms for Breast Cancer Detection and Identification of Key Predictive Features. *Trait. Signal* **2024**, *41*, 127–140. [CrossRef]
68. Al-Imran, M.; Akter, S.; Mozumder, M.A.S.; Bhuiyan, R.J.; Rahman, T.; Ahmmmed, M.J.; Mir, M.N.H.; Hasan, M.A.; Das, A.C.; Hossen, M.E. Evaluating Machine Learning Algorithms For Breast Cancer Detection: A Study On Accuracy And Predictive Performance. *Am. J. Eng. Technol.* **2024**, *6*, 22–33. [CrossRef]
69. Seger, I.S.; Hadi, A.M. Achieving High Accuracy in Breast Cancer Diagnosis with CNN. *Mars J. Tek. Mesin Ind. Elektro Dan Ilmu Komput.* **2024**, *2*, 107–116.

70. Papyan, V.; Han, X.Y.; Donoho, D.L. Prevalence of neural collapse during the terminal phase of deep learning training. *Proc. Natl. Acad. Sci. USA* **2020**, *117*, 24652–24663. [[CrossRef](#)]
71. Malingan, N. Tensorflow Pruning. 2023. Available online: <https://www.scaler.com/topics/tensorflow/model-pruning/> (accessed on 2 November 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.